

Fakultät für Informatik
Universität Duisburg-Essen

Bachelorarbeit

Readability of Source Code: A Repeated N-of-1 Study on Loops and Conditionals in Bash, PowerShell, and Python

Lukas Gruber

Matrikelnummer: 3100476

Studiengang: Angewandte Informatik - Systems Engineering

Essen, 25.08.2025

Betreuer: Stefan Udo Hanenberg

1. Gutachter: Prof. Dr. Volker Gruhn

2. Gutachter: Prof. Dr.-Ing. Lucas Vincenzo Davi

Abstract

Scripting languages are wildly used for task automation, where control flow structures such as loops and conditional statements form the backbone of these processes. This work examines how the choice of scripting language influences the readability of code containing these structures, measured through reaction time. Two controlled experiments employing repeated N-of-1 trials were conducted: the first compared code snippets featuring conditional statements, and the second focused on snippets containing looping constructs. The languages featured in the experiments were Bash, PowerShell, and Python. Across both experiments, Bash consistently resulted in the longest reaction times, whereas Python produced the shortest. In the first experiment, Bash was 55.3% slower than Python, while PowerShell was 29.7% slower. In the second experiment, Bash was 49.9% slower, and PowerShell was only 4.5% slower than Python. The results suggest that Python has the highest readability, followed by PowerShell, with Bash being the least readable.

Zusammenfassung

Skriptsprachen werden weit verbreitet für die Automatisierung von Aufgaben eingesetzt, wobei Kontrollstrukturen wie Schleifen und bedingte Anweisungen das Rückrat dieser Prozesse bilden. Diese Arbeit untersucht, wie die Wahl der Skriptsprache die Lesbarkeit von Code mit solchen Strukturen beeinflusst, gemessen anhand der Reaktionszeit. Es wurden zwei kontrollierte Experimente mit wiederholten N-of-1-Versuchen durchgeführt: Das erste verglich Codeausschnitte mit bedingten Anweisungen, das zweite konzentrierte sich auf Codeausschnitte mit Schleifen. Die in dem Experiment verwendeten Sprachen waren Bash, PowerShell und Python. In beiden Experimenten führte Bash konsequent zu den längsten Reaktionszeiten, wobei Python die kürzesten erzeugte. Im ersten Experiment war Bash 55.3% langsamer als Python, während PowerShell 29.7% langsamer war. Im zweiten Experiment war Bash 49.9% langsamer, und PowerShell war nur 4.5% langsamer als Python. Die Ergebnisse deuten darauf hin, dass Python die höchste Lesbarkeit besitzt, gefolgt von PowerShell, während Bash am wenigsten lesbar ist.

Contents

List of Figures	iv
List of Tables	v
List of Abbreviations	vi
1 Introduction	1
1.1 Research Question	1
1.2 Structure of this work	2
2 Background	3
2.1 Scripting Languages	3
2.2 Experiment Design	4
2.3 Statistical Methods	5
3 Related Work	7
3.1 Readability of Source Code	7
3.2 Mental Representation of Programs	8
3.3 Large Scale Studies on Code Quality of Languages	9
3.4 Summary	10
4 Commonalities between Preliminary Experiments	12
4.1 Initial Considerations	12
4.2 Measurement and Conduction	13
5 Comparison Operators Preliminary Experiment	14
5.1 Initial Considerations	14
5.2 Experiment Layout	15
5.3 Measurement and Conduction	16
5.4 Task Design	16
5.5 Analysis	17
5.6 Discussion	18
6 Conditionals Preliminary Experiment	19
6.1 Initial Considerations	19
6.2 Experiment Layout	20
6.3 Task Design	21
6.4 Measurement and Conduction	21
6.5 Analysis	22
6.6 Discussion	24

7	Loops Preliminary Experiment	25
7.1	Initial Considerations	25
7.2	Experiment Layout	26
7.3	Task Design	27
7.4	Analysis	28
7.5	Discussion	31
8	Conditionals Main Experiment	32
8.1	Initial Considerations	32
8.2	Experiment Layout	33
8.3	Task Design	33
8.4	Measurement and Conduction	34
8.5	Analysis	35
8.6	Threats to Validity	36
8.7	Discussion	41
9	Loops Main Experiment	42
9.1	Initial Considerations	42
9.2	Experiment Layout	43
9.3	Task Design	44
9.4	Measurement and Conduction	44
9.5	Analysis	45
9.6	Threats to Validity	53
9.7	Discussion	53
10	Conclusion	55
	References	56
A	Contents of the USB flash drive	60

List of Figures

1	Example code snippets for the preliminary experiment evaluating comparison operators	17
2	Code templates for the preliminary conditionals experiment	21
3	Example code snippets containing an error for the conditional preliminary experiment	22
4	Interaction diagram language * error	23
5	Code templates for the preliminary loops experiment	27
6	Example code snippets containing an error for the loops preliminary experiment	28
7	Interaction diagram method * error	30
8	Code templates for the main conditionals experiment	34
9	Example code snippets containing an error for each language	34
10	Interaction diagram background * language	38
11	Code templates for the main loops experiment	44
12	Example code snippets containing an error for the main loops experiment	45
13	Interaction diagram method * error	49

List of Tables

1	Experiment results for the conditional preliminary experiment . .	18
2	Experiment results for the conditional preliminary experiment . .	23
3	Experiment results for the loops preliminary experiment	29
4	Results for the main experiment on conditionals	37
5	Incorrect answers of participants per scripting language	38
6	Results per participant for Bash, PowerShell, and Python (with- out removing outliers)	39
7	Results per participant for Bash, PowerShell, and Python (remov- ing global outliers)	39
8	Results per participant for Bash, PowerShell, and Python (remov- ing outliers per participant)	40
9	Results of loops experiment on reaction time	46
10	Results of language on reaction time	47
11	Incorrect answers of participants per looping method	48
12	Results per participant without removing outliers	50
13	Results per participant removing outliers per participant	51
14	Results per participants on language	52
15	Results of loops main experiment on Pretask Reading Time . . .	52

List of Abbreviations

The next list describes several abbreviations that will be later used within the body of the document

ANOVA	analysis of variance
CI	confidence interval
CI/CD	continuous integration and continuous delivery
IQR	interquartile range
M	mean (arithmetic mean)
N	Number of data sets (subjects)
TRT	treatment

1 Introduction

Scripting languages are widely used to automate tasks that would otherwise require manual effort. Unlike general-purpose programming languages primarily aimed at building complex systems, scripting languages assume the presence of powerful components and focus on connecting and orchestrating them [25]. Among popular scripting languages, Bash, PowerShell, and Python are commonly employed for this purpose. Bash is a Unix shell widely used in Linux and other Unix-like environments for system administration and automation tasks. PowerShell, introduced by Microsoft in 2006, was designed for Windows administration and offers object-based pipelines with deep integration into the operating system. Python, in contrast, is a general-purpose programming language that has become a dominant scripting tool for automation, data processing, and web applications [6].

Although these languages are often associated with different environments, their usage frequently overlaps, which may require choosing one over the other. An important factor to consider in such decision is the readability of code written in these languages. In software engineering, the focus is often placed on technical factors rather than human factors. When human factors are considered, the evidence standard for decision based on them often do not meet the standard conventions in empirical study design [32]. Therefore, the goal of this work is to conduct an empirical study using controlled experiments to compare the readability of source code in Bash, PowerShell, and Python, with a focus on loops and conditionals. The application of empirical methods is essential for making software engineering more objective and less imprecise [13].

1.1 Research Question

This work investigates if the scripting language affects readability and error detection in code. The scripting languages examined are Bash, PowerShell, and Python, with a focus on loops and conditionals in short code snippets. In this context, readability and error detection refer to how quickly a reader can read code and determine whether it is valid—for example, whether it would cause an error when executed or change the intended logic of the code. Controlled experiments are conducted to evaluate the three languages, with the hypothesis that the differences in syntax and structure will affect reader performance.

Across both of our main experiments, Bash consistently resulted in the longest reaction times, whereas Python produced the shortest. In the first experiment, Bash was 55.3% slower than Python, while PowerShell was 29.7% slower. In the second experiment, Bash was 49.9% slower, and PowerShell was 4.5% slower than Python. The results revealed that the scripting language used

did have a significant effect on the readability of code ($p < .001$ for both experiments).

1.2 Structure of this work

This work is structured as follows: Section 2 provides background information relevant to the study. Section 3 reviews related work, focusing on experiment design and scripting languages. Section 4 identifies commonalities across preliminary experiments and presents initial considerations and details on the design and conduction of the experiments. Sections 5, 6, and 7 present the design and analysis of the preliminary experiments. Section 8 and 9 describe planning, designing, execution, and analysis of the two main experiments on conditionals and loops. Section 10 summarizes the findings and concludes this work.

2 Background

This section provides the necessary background for the study, including the scripting languages employed and their support for loops and conditionals. It also outlines experimental designs commonly used in empirical research and the statistical methods applied to evaluate the findings.

2.1 Scripting Languages

Ousterhout [25] notes that scripting languages are designed for different tasks than conventional programming languages. Instead of building complex components from scratch, they assume the presence of powerful components and focus on connecting and orchestrating them. Although the distinction between traditional programming languages and scripting languages is often blurred, a defining feature of scripting languages is their dynamic nature, which enables data to be treated as a program and programs to be manipulated as data [18]. Additionally, scripting languages are typically interpreted, whereas system programming languages are usually compiled [25, 18].

An important feature in scripting languages is the availability of control flow structures, such as loops and conditionals, which enable repeated execution of tasks and automated decision-making. Because the syntax and implementation of these structures differ between scripting languages, they can potentially affect the readability of scripts.

Bash Bash is the command-line interpreter—also called shell—used in the GNU operating system. Its name stands for “Bourne-Again Shell”, a reference to Stephen Bourne, who created `sh`, the Unix shell that Bash was designed to replace [27, p. 1]. In Unix systems, the shell acts as both a command interpreter and a programming language, translating user-entered commands into instructions for the operating system [24, p. 1][27, p. 1]. In Bash, common loop constructs include `for`, `while`, and `until`, while conditional statements are implemented using structures like `if`, `elif`, and `else`. Chaining can be achieved using symbolic logical operators (e.g. `&&` and `||`), whereas comparisons can be performed with both symbolic (e.g., `<`, `>`, and `==`) and word-like operators (e.g., `-lt`, `-ge`, and `-eq`) [2].

PowerShell PowerShell is a versatile automation tool that works across different operating systems. It combines command-line interface, a scripting language, and tools for managing system configurations, and it is available for Windows, Linux, and macOS. PowerShell is frequently used as a scripting language to automate system management. It is also utilized in the development, testing, and deployment of software solutions, particularly within CI/CD pipelines. PowerShell is implemented on the .NET

Common Language Runtime (CLR) [23]. In his book, Shepard [30, p. v] notes that PowerShell has become the standard scripting solution for Windows since its introduction, enabling automation of a wide range of tasks across the Windows ecosystem.

PowerShell supports multiple looping constructs such as `for`, `foreach`, or `ForEach-Object` [22]. Conditionals are implemented using `if`, `elseif`, and `else`, with comparisons expressed through word-like operators (e.g., `-lt`, `-ge`, and `-eq`). Logical operators are also word-like and, similar to the comparison operators, prefixed with a hyphen [21].

Python Python is an interpreted, high-level programming language whose design philosophy emphasizes code readability, notably through the use of significant indentation to structure code [6]. It consistently ranks among the most widely used programming languages [8, 5, 7]. In addition to general-purpose programming, Python is frequently employed as a scripting language for web applications and is embedded in numerous software products [6].

Similar to other languages, Python offers multiple control flow tools. Loops are created using statements such as `for` and `while` [1], while conditionals are implemented using the `if`, `elif`, and `else` keywords. Python uses symbolic operators (e.g., `<`, `>`, and `==`) for comparisons and word-like operators (`and`, `or`, and `not`) for boolean logic.

Unlike many other programming languages, Python does not use braces or other explicit delimiters to indicate code blocks, it uses indentation. Each block of code—such as those associated with conditional statements or loops—must be indented. Otherwise, Python raises a runtime error.

2.2 Experiment Design

In software engineering, empirical studies can help change how software is developed [35, p. 6]. However, few ideas are backed by empirical evidence. Unverified assumptions are often treated as self-evident truths [17, p. 4].

Controlled experiments investigate how deliberately changing independent variables influences one or more dependent variables. They make it possible to determine, with precision, the nature of the relationship between variables and whether a cause-effect relationship exists. Every unique combination of independent variable values forms a treatment [31, p. 294]. Dependent variables are measured to evaluate the impact of changes in the independent variables. Likewise, independent variables are systematically manipulated within the experiment and are expected to influence the dependent variables [31, p. 217].

A central aspect of controlled experiments is the experiment design, which determines how participants are assigned to treatments.

Randomized Controlled Trials In conventional randomized controlled trials (RCTs), which typically employ a between-subjects design, each participant is randomly assigned to a single treatment. This process is intended to ensure the characteristics of the participants are comparable across the study groups. Randomization mitigates the risk of imbalance in both known and unknown factors [33].

Within-Subject Design In contrast to between-subject designs, within-subject designs involve each participant receiving multiple treatments [11]. This allows the direct comparison of treatments within the same individual, while usually requiring a smaller sample size than between-subject designs [9]. N-of-1 studies are a specialized type of within-subject design in which a small number of participants, typically one, receives multiple treatments [14]. However, with within-subject designs, the order of exposure can affect participants' responses to treatments due to learning effects, novelty effects, or fatigue effects, making it an important factor to consider when designing the experiment [11, 14].

2.3 Statistical Methods

When conducting an experiment, statistical methods are required to evaluate the findings and determine whether observed differences are statistically significant or due to chance.

ANOVA Analysis of variance is a statistical method used to determine whether there are statistically significant differences between the means of multiple groups, by comparing the variance between groups to the variance within groups [35]. It can be applied to experiments with one or more independent variables and is commonly used when testing the effect of categorical factors on a continuous outcome. To assess statistical significance, ANOVA calculates a p-value. If this p-value is below a predetermined α level, the result is considered statistically significant. This work adopts $\alpha = .05$ as the significance threshold, as this threshold is commonly used in empirical research, e.g. by [10, 20, 16]. The effect size can be expressed using the partial eta squared (η_p^2), which represents the variance attributed to the effect and dividing it by the combined variance from the effect and the residual error [20]. Partial eta squared values of approximately .01, .06, and .14 indicate small, medium, and large effect sizes as stated by Richardson [29].

Post Hoc Tests When an ANOVA shows a statistically significant result, it indicates that at least one group mean differs from the others, but it does not specify which groups differ. Post hoc tests are then carried out to compare all possible pairs of group means [10].

Chi-square Test According to Khanfar et al. [19], the chi-square (χ^2) statistical method is employed to assess whether the distribution of categorical variables show significant differences. The test works only with raw numerical counts, not with calculated percentages, proportions, means, or similar.

Interquartile Range The interquartile range (IQR) technique can help detect outliers in a data set. The IQR measures the spread of the middle 50% of data and is calculated as the difference between the first (Q1, the 25th percentile) and third quartile (Q3, the 75th percentile): $IQR = Q3 - Q1$. Data outside of $[Q1 - 1.5IQR, Q3 + 1.5IQR]$ are considered outliers [34].

jamovi - Jamovi is an open-source statistical analysis software that provides a graphical user interface for performing a wide range of statistical procedures [4]. This work uses jamovi 2.6.26.

3 Related Work

This section reviews five empirical studies relevant to this work: two on code readability, one on the mental representation of programs, and two large scale analyses of GitHub repositories examining, among other things, bugs related to programming languages.

3.1 Readability of Source Code

The following two studies investigate source code readability through controlled experiments measuring factors such as reading time, response time, and accuracy. Both studies also propose models for quantifying readability.

Johnson et al.: An Empirical Study Assessing Source Code Readability in Comprehension [16]

Johnson et al. investigated how specific coding practices influence a developer's ability to read and understand code. Their study focused on the Java programming language and examined two readability rules: minimizing the nesting of control structures and avoiding the use of do-while loops. A within-subject controlled experiment was conducted involving 275 participants who were given 32 Java code snippets. Each code snippet was categorized into whether they follow the readability rule and whether the code snippet was correct, resulting in four treatments. The participants were asked to read the code snippets, answer questions on readability and comprehension, and answer whether the code snippet was correct. Additionally, the completion time was measured by two metric: the time to read the code snippet, and the time to answer the questions.

The findings of the study revealed that minimizing nesting significantly improved comprehension ($p = .0495$, Cohen's $d = .1224$) and reduced the time to read code ($p < .0001$, Cohen's $d = -.4807$) and answer questions regarding the code ($p = .0602$, Cohen's $d = -.1171$). Avoiding the use of do-while loops showed no significant impact on any of the metrics collected.

In regards to reading times, Johnson et al. conclude that minimizing the nesting of control structures has the potential to reduce reading times. For the second readability rule, avoiding the use of do-while loops, they state that following the rule saves time only for short and simple loops.

Hanenberg et al: Indentation and reading time: a randomized control trial on the differences between generated indented and non-indented if-statements [15]

In their work, Hanenberg et al. examined the effect of indentation on source code readability and presented a formal model to explain differences between indented and non-indented code. Based on this model, they designed a controlled experiment using automatically generated Java code snippets composed of nested if-statements. The experiment was conducted as a repeated N-of-1 study with 27 participants. Each participant was asked to determine the output of presented code snippets for indented and non-indented code in 90 tasks. For each task the reaction time was measured until a participant gave their answer.

An analysis of the results of the experiment revealed a strong and large effect of indentation ($p < .001$, $\eta_p^2 = .198$), with non-indented code requiring, on average, 113% more time to read. Additionally, the experiment showed that the more code can be skipped in indented form, the larger the differences in reading time become ($p = .001$, $\eta_p^2 = .072$). However, analyses of individual participants could only confirm this for about half of the participants (13/27).

Hanenberg et al. concluded that indentation significantly improves code readability and that differences in reading times can be explained by the amount of skippable code in indented versions. However, they acknowledge that not all potential influencing factors were controlled in their experiment, leaving room for future studies to investigate additional variables affecting readability of code.

3.2 Mental Representation of Programs

The following study examines how expert programmers mentally represent programs, providing insights into participants' behavior when first reading code and memorizing key aspects.

Pennington: Stimulus Structures and Mental Representations in Expert Comprehension of Computer Programs [26]

In their work, Pennington investigated how expert programmers comprehend computer programs by investigating whether procedural (control flow) or functional (goal hierarchy) relations dominate their mental representations of programs. For that, Pennington designed two empirical studies. In the first experiment, 80 professional programmers (40 COBOL programmers and 40 FORTRAN programmers) were asked comprehension questions (yes or no questions) targeting either procedural aspects or functional aspects of 15 lines long code snippets—eight code snippets written in COBOL and FORTRAN. For each

of the eight program segments, six comprehension questions were created, each targeting a specific category of information about the relationships within the program, which the participants had to answer from memory after studying the code.

The response times and correctness of the answers were analyzed, showing that responses to procedural-primed targets were on average 105 ms faster than responses to functional-primed targets ($p < .04$). Participants answered questions about detailed operations and control flow relations more accurately, with error rates of 15% and 21%, respectively, while higher error rates were observed for items on data flow, state, and function, at 28%, 30%, and 34%, respectively.

Following the first study, Pennington conducted a second study in which 40 professional programmers—who all participated in the first study, including 20 COBOL programmers and 20 FORTRAN programmers—were asked to study a 200 line program and then modify it. The participants were given time to study the code, after which they answered 20 questions on control flow, data flow, function, and program state. Then they modified the code and subsequently answered 20 additional questions of similar type.

On the initial questions, the subjects made more errors on questions regarding function and program state ($p < .001$). However, on the second set of questions, after modifying the code, this behavior shifted significantly ($p < .001$), with participants making more errors on control flow questions ($p < .001$).

Pennington concluded, that comprehension results from the second study support and extend the findings from the first study, indicating that understanding of program control flow precedes comprehension of program functions.

3.3 Large Scale Studies on Code Quality of Languages

The following two studies present large scale analyses of multiple GitHub repositories. The first study examines the impact of programming language choice on the frequency of software defects. The second study focuses on common language usage, code smells, and bugs found in Bash scripts, highlighting common issues and complexities associated with this language.

Ray et al.: A Large Scale Study of Programming Languages and Code Quality in Github [28]

In their work, Ray et al. investigated the effect of the programming language on software quality. For that, they analyzed data from 729 GitHub projects, 80 million lines of code, 29000 authors, and 1.5 million commits across 17 programming languages. To analyze if some languages are more defect prone than others, the number of defective commits was analyzed, showing that while the

effect is statistically significant for some languages, the effects are quite small. They found that C, C++, JavaScript, Objective-C, Php, and Python tend to have more bugs, while Clojure, Haskell, Ruby, Scala, and TypeScript tend to have fewer. Grouping the languages by language class, they also found that functional languages produce fewer bugs than procedural and scripting languages ($p < .001$).

They conclude that while the programming language choice had a measurable effect on software quality, the magnitude of this effect was relatively small. However, they note that the large number of dependent variables in their study makes it difficult to quantify these effects.

Dong et al.: Bash in the Wild: Language Usage, Code Smells, and Bugs [12]

Dong et al. conducted a large-scale empirical study of Bash usage by analyzing over one million open source Bash scripts found in GitHub repositories, providing insight into commonly found language features, code smells, and bugs.

Their analysis of language features revealed that loops and arrays are rarely used, suggesting that Bash scripts tend to follow a linear structure with occasional conditional branches. They also observed that utilities related to files, paths, and directories are among the most frequently used.

To identify code smells in Bash scripts, the researchers used the ShellCheck static analysis tool, which detects various syntactic and semantic issues in shell scripts. Their findings indicate that Bash scripts are particularly prone to code smells, even in top repositories. The most common issues involved quoting and word splitting (found in 20.9% of files), error handling (21.5%), and return values (8.7%).

To find common bugs, the researchers randomly sampled 200 bug-fixing commits and manually inspected them. They discovered 57 Bash semantic bugs, 122 application semantic bugs, 3 Bash syntactic bugs, and 32 false positives. Among the 57 Bash semantic bugs, 15.7% were related to quoting, 14.0% to file, path, and directory management, and 10.5% each to command options, permissions, and error handling.

Dong et al. conclude that Bash scripts are error-prone, with many issues potentially going undetected. In their samples, only 20% of Bash scripts were free of code smells. They recommend that Bash practitioners adopt defensive programming practices, such as checking command exit statuses and quoting variables to prevent unintended word splitting.

3.4 Summary

The first two papers investigated the readability of source code by conducting controlled experiments, using a within-subject experiment design. The first

study by Johnson et al. measured the time participants took first to read a code snippet and then to answer related questions. The second study by Hanenberg et al. presented participants with a task and a code snippet, measuring the time taken to complete the task. Both studies assessed readability by quantifying how long participants took to understand the code.

The second study by Pennington investigated participants' behavior when first reading code, memorizing key aspects, and then answering related questions. The found found that participants initially made more errors on questions about function and program state than on control flow questions. After participants had time to modify the code, this pattern shifted, with more errors occurring on control flow questions.

Ray et al. found, through a large scale analysis, that the programming language had a small but significant effect on software quality. In their data set, functional languages produced fewer bugs than procedural and scripting languages.

Also using a large scale analysis of GitHub repositories, the final study by Dong et al. examined multiple Bash scripts and found that many contained code smells or bugs. They conclude that Bash scripts are prone to errors, with only 20% of Bash scripts in their sample being free of code smells.

4 Commonalities between Preliminary Experiments

Multiple preliminary experiments were created with the common goal to ultimately building a main experiment comparing the readability of source code in relation to Bash, PowerShell, and Python. With this, these preliminary experiments share commonalities in terms of initial consideration, experiment design, and conduction. These commonalities are described in this section.

4.1 Initial Considerations

Each experiment compares programming languages by creating code snippets for the participant. To compare the languages, we want to measure the reading time for each of those code snippets. For this purpose, we measure the reaction time, i.e., the time it takes for the participant to complete a task, which serves as an approximation of reading time, similar to Hanenberg et al. [15]. In addition to that, we measure whether the participant's given answer is correct.

Designing an experiment to compare Bash, PowerShell, and Python goes hand in hand with the participants chosen for the experiment. Ideally, the experiment would involve participants who are proficient in all three languages. However, finding participants with those qualifications is challenging. To address this limitation, the experiment design must accommodate varying levels of language proficiency. A training phase should familiarize participants with the task required for the experiment. Consequently, the experiment must be performable for people with general programming experience but little to no knowledge in any of the three languages.

Task To reduce the complexity of each task, we do not want the participants to write code themselves. The tasks can be to either identify valid or invalid code or to give the expected output of the code snippet.

N-of-1 N-of-1 trials, in contrast to between subject experiments, give all participants all treatment combinations. This makes it possible to study the effect on single participants.

Participants Optimally, the participants have programming knowledge in all three languages, Bash, PowerShell, and Python. For the experiments we opted to reduce the requirements to basic knowledge in programming and scripting languages. As the participants do not need to write code themselves, which would require more advanced knowledge in the respective languages, the participants are only required to have basic code knowledge in these languages.

The author of the paper was the only participant for all of the preliminary experiments.

Training Participants Before we conduct the experiment, the participant receives a training set of code snippets similar to the main experiment. This aims to reduce the influence of learning effects on measured times in the experiment phase, as the participant can learn with the training set. Reading times are not measured for the training set, which may be repeated if chosen by the participant. The participant only continues to the main experimentation phase if they feel fit to do so. The training aims to teach the basic structure of loops and conditionals for each language. It should provide enough knowledge to solve the tasks in the experiment but does not aim to teach the languages beyond that.

4.2 Measurement and Conduction

Task may vary in length, and the estimated time to complete some tasks may be short. The method to measure the time must be correspondingly fast. Therefore, measurement is done by using a single key press on a keyboard. An interactive interface [3] was used to generate and present the code snippets as plain text, without syntax highlighting. In the experiment, the participant then completes each task with a key press, representing the answer to the task. As the tasks vary, the necessary keys to press also vary with each experiment. With that method, only little time is spent from when the participant identifies the answer to the task to inputting the result. After pressing the key, the answer, result, and times are measured and saved. The participant can then, by pressing `[Return]` change the view to the next task. The reaction time in milliseconds is measured from the moment the participant switches to the task (by pressing `[Return]`), to when an answer is given. The time is measured for each task individually and the timer restarts for each task. The times and answers are only measured for the main experiment and not during the training phase. The participants were allowed to take breaks between tasks.

After the experiment, a csv-file is generated which include all the measured results.

5 Comparison Operators Preliminary Experiment

There are many structural differences between programming languages, which can impact how developers read, write, and understand code. One such structural difference lies in how comparison operators are used in conditional statements. Since these operators are both fundamental to program logic and vary notably in syntax and semantics between languages, they can be a source of confusion and errors. Bash uses word-based operators for numeric comparisons (e.g., `-eq`, `-lt`, `-gt`), whereas Python uses symbolic operators (e.g., `==`, `<`, `>`). This preliminary experiment investigates how such differences affect code comprehension and error detection by comparing short code snippets written in Bash and Python. As PowerShell uses numeric comparison operators similar to Bash, it is excluded from this experiment.

5.1 Initial Considerations

We want to design a controlled experiment where the participant receives code snippets measuring the effect of different comparison operators. To compare the programming languages, we want to measure the reading time for each code snippet. For that we measure the reaction time, i.e., the time it takes for the participant to complete a task, which serves as an approximation for the reading time. In addition to that, we measure whether the participants' given answer is correct.

Comparison Operators The scope of this experiment is limited to comparison operators used for comparing integer values. This specific focus was chosen because Bash and Python use distinct styles of comparison operators in this context. Bash employs a syntax commonly found in shell and scripting languages (e.g., `-eq`, `-lt`, `-gt`), whereas Python uses symbols (e.g., `==`, `<`, `>`) similar to those in many high-level programming languages.

Error Detection The scope of the experiment is to understand the difficulties of error detection in the different languages, errors should be introduced into the tasks. As this experiment focuses on comparison operators within conditional statements, only these comparison operators should include the errors. This prevents participant from wasting time searching for errors in other parts of the code, such as the condition body.

Task Preparation Switching between the different programming languages during the experiment may be confusing and cost reading time. Therefore, each code snippet starts with a comment naming the programming language used.

Tasks As we focus on code comprehension, the program can have different results based on the conditional statement. We can ask the participant for the output of the program. This output depends on if the conditional statements containing the comparison operators is true or false, which ensures that the participant has comprehended the code snippet. We want to prevent participants from focusing on the condition body so that we only measure the time it takes to read and comprehend the conditional statement. Therefore, the condition body should be simple and the output should not change, i.e., the output for if the condition is true should be consistent and the output if the condition is false should also be consistent. To include the error detection into the task, some code snippets may contain errors as already described. In conclusion, the participant should identify if the conditional statement in the code snippet is true, false, or contains an error.

Code Length Differences in code length can greatly increase or decrease the time it takes to solve a task. To avoid this, each task in each language, should have the same number of lines of code. Another consideration is the number of conditional statements in each task. We decided to have each task only contain a single conditional statement to reduce the task length as much as possible.

Code Snippets Each snippet should contain a valid conditional statement with one `if` clause and one `else` clause. When the condition is true, the `if` clause should output "1"; when it is false, the `else` clause should output "2". For each task, only the condition should change. In the condition two randomly generated small numbers should be compared using the respective comparison operators for each language.

While the code snippets contain more logic than just the condition in the conditional statement, we expect the participants to only consider this code in the training phase. During the main experiment, we expect the subject to only determine whether the comparison operator is valid, identify the two numbers, and finally compare them using the comparison operator.

5.2 Experiment Layout

As previously discussed, the experiment is designed as a crossover N-of-1 experiment with only one participant. The participant receives 200 randomly generated code snippets. The experiment layout consists of the following parameters:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his result. This variable should approximate the reading time.

- **correctness:** Whether the given response is correct or not.
- **Independent Variables:**
 - **language:** The programming language used for the task. For this variable there are two treatments, either Bash or Python.
 - **error:** Whether the comparison operator used in the task is valid or not. There are two treatments for this, either valid or invalid.
- **Fixed Variables:**
 - **repetitions:** The experiment repeats each treatment combination 50 times for a total of 200 code snippets.
- **Task:** *“Press [1] if the conditional is true, [2] if the conditional is false, and [e] if the conditional is invalid.”*

The keys [1] and [2] represent the output of the condition body and [e] represents an invalid configuration.

The goal of the experiment is to see if the independent variables influence the dependent variables.

5.3 Measurement and Conduction

As discussed in Section 4.2, the experiment is executed by an application which also measures the results. In this experiment the participant has to press either “1”, “2”, or “e” on the keyboard to complete each task. These key presses are used by the application to measure the reaction time.

5.4 Task Design

In the experiment each task presents the participant with a combination of the values for the independent variables. Each task uses either Bash or Python containing a conditional statement with a single integer comparison. To introduce an error into the task, the correct comparison operator is intentionally replaced with a syntactically similar but incorrect one.

Figure 1 shows an example code snippet using Bash. It starts with the comment naming the programming language used. The next line marks the beginning of the conditional construct, including its condition and the comparison operator used, in this case the `-ge` operator, which stands for “greater or equal”. All valid Bash comparison operators used in the experiment are `-lt`, `-gt`, `-le`, `-ge`, `-eq`, and `-ne`. The invalid operators used in the experiment code are `-ls`, `-gs`, `-gr`, `-eg`, and `-nq`. These incorrect operators are designed to resemble the valid ones in structure to increase the difficulty of immediate error detection. For example, `-ls` closely resembles `-lt`, but is not a valid Bash

```
# Bash
if [ 7 -ge 8 ]; then
    echo "1"
else
    echo "2"
fi

# Python
if 7 >= 8 :
    print("1")
else:
    print("2")
```

Figure 1: Example code snippets for the preliminary experiment evaluating comparison operators

operator. The `echo "1"` represents the output if the condition is true, and the `echo "2"` represents the output if the condition is false.

Figure 1 also shows an example code snippet using Python. It is structurally similar to the Bash code and also starts with a comment naming the programming language used. Then, the conditional construct begins, including its condition and the comparison operator `>=`, which also stands for “greater or equal”. The valid Python comparison operators used in the experiment are `<`, `>`, `<=`, `>=`, `==`, and `!=`. Here, the incorrect operators are also designed to resemble the valid ones. The invalid operators used in the code snippets are `<<`, `>>`, `=<`, `=>`, `=`, and `=!`. The `print("1")` and `print("2")` represent the output depending on whether the condition is true or false.

In each task, both numeric values and the operators are randomly varied. This causes the the condition to sometimes be true, false, or cause an error if an invalid operator is used.

5.5 Analysis

The measured reaction times were analyzed using an ANOVA and the number of errors were analyzed using a χ^2 test . The χ^2 test was performed on the independent variable language and the dependent variable correctness.

Looking at Table 1, we see that overall only a few errors were made. Out of 200 responses, 5 were incorrect, resulting in an error rate of just 2.5%. The errors were almost evenly distributed across the languages, with 2 incorrect responses in Bash and 3 in Python. This difference is statistically insignificant ($p = .651$, see Table 1).

The ANOVA in Table 1 shows a statistical significance of the independent variable language ($p < .001$) as well as a statistical significance of the indepen-

Table 1: Experiment results for the conditional preliminary experiment

ANOVA on Reaction Time (approximator for Reading Time)							
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M
language	1	32.23	< .001	.141	Bash	1310; 1477	1394
					Python	1048; 1202	1125
error	1	91.85	< .001	.319	no error	1399; 1573	1486
					error	980; 1085	1033
language * error	1	1.35	.247	.007	Bash/no error	1544; 1752	1648
					Bash/error	1053; 1226	1139
					Python/no error	1198; 1450	1324
					Python/error	882; 969	926

χ^2 -Test on language						
Variable	df	χ^2	p	N	TRT	Incorrect Answers
language	1	.205	.651	100	Bash	2
				100	Python	3

dent variable error ($p < .001$). In addition, both independent variables measured a large effect, with language having a η_p^2 of .141 and error having a η_p^2 of .319. However, the table also shows that there is no interaction effect of language and error ($p = .247$).

The analysis shows that tasks using Python were completed more quickly than those using Bash. This suggests that Python’s symbolic comparison operators—such as `>=`—are faster to read and interpret than Bash’s typical operators like `-ge`. Although, it is important to note that all the measured times were very small with the confidence intervals ranging from 1048 milliseconds to 1202 milliseconds for Python and 1310 milliseconds to 1477 for Bash (see $CI_{95\%}$ in Table 1). The means for Bash were 24.1% larger than the means for the Python code.

5.6 Discussion

The experiment shows that while Python’s symbolic comparison operators did not lead to any significant change in correctness of answers ($p = .651$), a significant effect on reaction times was observed ($p < .001$). The effect size was large with a η_p^2 of .141. However, although the mean reaction time for Bash was 24.1% larger than that for Python, the absolute difference was relatively small, with task completion times of 1394 milliseconds and 1125 milliseconds, respectively. A follow-up experiment comparing larger parts of the conditional control flow structure in these languages might reveal whether these differences persist or whether looking at the whole conditional structure leads to different results. Furthermore, replicating this experiment with participants of different demographics and programming background could clarify whether reaction times for symbolic comparison operators persist to be smaller.

6 Conditionals Preliminary Experiment

This preliminary experiment focuses on conditionals in Bash, PowerShell, and Python. Unlike the previous preliminary experiment that only focused on comparison operators used in conditionals; this experiment aims to distinguish differences in code comprehension and error detection for the whole conditional structure used in the three aforementioned programming languages. Examining the entire conditional structure is important because conditionals vary not only in their operators but also in syntax, keywords, and overall format, all of which can influence how easily developers understand and correctly implement logic across different languages.

6.1 Initial Considerations

We want to design an experiment that presents code snippets to the participants. These code snippets should show representations of conditional statements in the respective language. As done in the previous preliminary experiment (see Section 5), we want to measure the reading time for each code snippet by measuring the time it takes for the participant to complete a task.

Error Detection Similar to the previous preliminary experiment, the scope of this experiment is to compare error detection in regards to the programming language used. Therefore, we want to introduce errors into the code snippets. In the comparison operators preliminary experiment (see Section 5) we only introduced errors to the comparison operators in a conditional statement. For this experiment, we want to widen our focus on the structure of the full conditional statement. This means that we ideally want the participant to read every token of the statement, i.e., every keyword and every symbol. To achieve this, the code snippets should have errors randomly in different positions, possibly at every token of the code. For this experiment, we define an error as a modification of the code that would cause an exception when running the code or alter the way the code behaves in an unexpected way. The errors introduced should differ for every token respectively and modify it in a way to cause an error and ideally mimic some of the errors a developer could easily introduce when writing code, e.g. typos like `fi` instead of `if`, or using the wrong brackets like `(...)` instead of `[...]`. This results in the participant having to look at every token of the code snippet to find the modified keyword or symbol.

Task As we focus on error detection with this experiment, the participant should respond, whether the given code snippet includes an error or not. Therefore, we want to give the participants some code snippets that include an error as described earlier, and some that do not include an error.

We deliberately decided to not have the participants write any code themselves to further reduce the complexity of the tasks.

Task Preparation In the previous preliminary experiment, code snippets began with a comment naming the programming language used. Determining the language from the comment and preparing to work with it could still require additional time during the task. Therefore, for this experiment, the language should be provided before the task. This removes the time needed to deduce the language from the code snippet and allows participants to mentally prepare for the task.

Code Snippets Since we do not want the participants to require advanced programming knowledge in all the languages, we want to reduce the complexity of the code snippets. We decided to only include one conditional statement into each code snippet for each task. The code snippet only contains the conditional statement with a simple condition body, which never contains an error and prints “. . .” in the respective language. Each condition consists of one string comparison and one numeric comparison, combined into one statement by a logical `and` operator. While combining two conditions increases the complexity of the code, it also introduces more potential for errors. Additionally, it allows us to examine both string comparisons and numeric comparisons. To avoid further increasing the complexity of the code, the combination of the two conditions is always done using the `and` operator.

See further considerations common in all preliminary experiments in Section 4.1.

6.2 Experiment Layout

The experiment layout consists of the following variables:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his result. This variable should approximate the reading time.
 - **correctness:** Whether the given response is correct or not.
- **Independent Variables:**
 - **language:** The programming language used for the task. For this variable there are three treatments, either Bash, PowerShell, or Python.
 - **error:** Whether the conditional statement contains an error or not.
- **Fixed Variables:**

- **repetitions:** The experiment repeats each treatment combination 30 times for a total of 180 code snippets.
- **Task:** *“Press [1] if the conditional statement is valid and [e] if the conditional statement is invalid and contains an error.”*

Again, the goal of the experiment is to see if the independent variables influence the dependent variables.

Bash:

```
if [[ "$name" == "Max" && $age -ge 18 ]]; then
    echo "..."
```

PowerShell:

```
if ($name -eq "Max" -and $age -ge 18) {
    Write-Output "..."
```

Python:

```
if name == "Max" and age >= 18:
    print("...")
```

Figure 2: Code templates for the preliminary conditionals experiment

6.3 Task Design

For each task, the code snippets are created from one of three templates. For each language, the templates correspond to the code snippets shown to participants when the conditional statement contains no error (see Figure 2). In the *correct* snippets, all tokens are static and presented exactly as depicted in the figure. For snippets that contain an error, a single token is randomly selected and replaced with a predefined error intended to resemble those that a developer might introduce when writing code. Figure 3 presents an example code snippet for each language containing an error. The Bash snippet is invalid because it omits the `$` when referencing the `name` variable. The PowerShell code is invalid because it is missing the `-` for the `-and` logical operator. Finally, the Python snippet is invalid because it omits the quotation marks around `Max`.

6.4 Measurement and Conduction

The measurement and conduction is done as described in Section 4.2 with some adjustments. For this experiment, the participants can solve the tasks by pressing either [1] or [e], as described in Section 6.2. After each task, the [Return] key does not immediately switch to the next task, but to the

Bash:

```
if [[ "name" == "Max" && $age -ge 18 ]]; then
    echo "... "
fi
```

PowerShell:

```
if ($name -eq "Max" and $age -ge 18) {
    Write-Output "... "
}
```

Python:

```
if name == Max and age >= 18:
    print("... ")
```

Figure 3: Example code snippets containing an error for the conditional preliminary experiment

task preparation view (see Task Preparation in Section 6.2). Another [Return] press, changes to the next task, whenever the participant feels prepared.

6.5 Analysis

Looking at the χ^2 -Test in Table 2, we see that all three languages attributed to some errors. In total, out of 180 responses the participant answered 161 correctly and 19 incorrectly. This results in an overall error rate of around 10.6%. For Bash, the error rate is the highest with around 13.3% and 8 incorrect responses. For PowerShell, the error rate is the lowest with around 6.7% and 4 incorrect responses. The error rate for Python is in between those two with around 11.7% and 7 incorrect responses. This difference is statistically insignificant with $p = .465$.

The ANOVA in Table 2 shows a statistical significance of the independent variable language ($p < .001$) as well as a statistical significance of the independent variable error ($p < .001$). The language used also measured a medium to large effect with a η_p^2 of .135. The independent variable error measured an even higher effect with a η_p^2 of .176. However, the ANOVA shows no significant interaction effect of language and error ($p = .066$), though the result approaches significance. Repeating the experiment with a larger sample size or different participants may reveal an interaction effect; however, in this experiment, the effect size is small with a η_p^2 of .031.

The experiment shows that there is a significant difference in reaction time when using different programming languages for conditionals. The mean of the measured times for Bash was 3709 milliseconds, which is more than 19% slower than the means for PowerShell and Python—2976 milliseconds and 2813 milliseconds respectively. Figure 4 visualizes the interaction between the lan-

Table 2: Experiment results for the conditional preliminary experiment

ANOVA on Reaction Time (approximator for Reading Time)							
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M
language	2	13.61	< .001	.135	Bash	3393; 4026	3709
					PowerShell	2682; 3271	2976
					Python	2570; 3056	2813
error	1	37.12	< .001	.176	no error	3448; 3795	3621
					error	2442; 2980	2711
language * error	2	2.76	.066	.031	Bash/no error	3871; 4504	4187
					Bash/error	2722; 3740	3231
					PowerShell/no error	3406; 3861	3634
					PowerShell/error	1880; 2758	2319
					Python/no error	2827; 3259	3043
					Python/error	2148; 3019	2584

Post Hoc Test			
Variable	Variable	df	p
Bash	PowerShell	174	< .001
	Python	174	< .001
PowerShell	Python	174	.647

χ^2 -Test on language						
Variable	df	χ^2	p	N	TRT	Incorrect Answers
language	2	1.53	.465	60	Bash	8
				60	PowerShell	4
				60	Python	7

guage and error. Since the means for PowerShell and Python are close, a post hoc test was conducted, confirming no significant difference between the two languages (see $p = .647$ in Table 2).

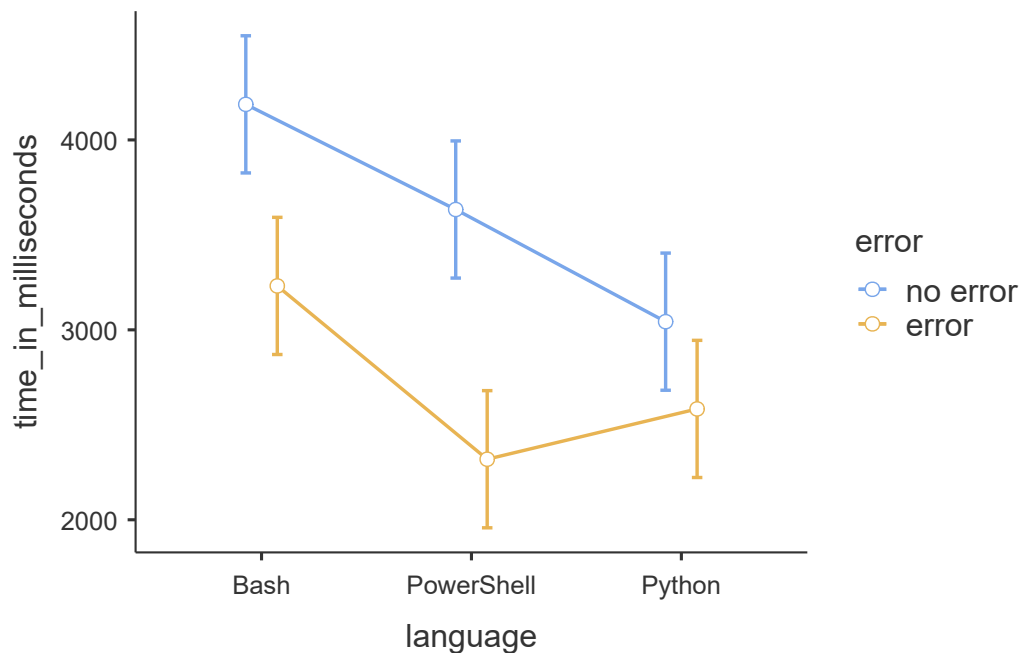


Figure 4: Interaction diagram language * error

6.6 Discussion

The results show a medium to large significant effect of the language used on measured reaction times ($p < .001$, $\eta_p^2 = .135$). A post hoc analysis revealed that this effect was driven by Bash, which differed significantly from the other languages, whereas no significant differences were observed between PowerShell and Python ($p = .647$). While we confirmed significance of the scripting language regarding the readability of conditional statements, no significant effect was found for error detection ($p = .465$), with no language resulting in an abnormally high number of errors. These findings suggest that the choice of scripting language can influence readability, but the limited sample size means that the results should be interpreted with caution. While the tasks were randomly ordered, order effects could still be present. With multiple consecutive tasks occasionally using the same programming language, carryover effects, fatigue, or boredom could occur. Repeating the experiment with a larger number of participants and a different order could help confirm whether these effects are consistent across different individuals and minimize the risk of unwanted order effects.

7 Loops Preliminary Experiment

While the previous two experiments focused on conditional statements in scripting languages, this experiment addresses another fundamental control flow construct of programming: loops. Loops are essential to programming as they allow developers to execute instructions repeatedly under specified conditions. As with conditionals, the syntax and semantics of loops differ between languages, which could potentially affect how easily developers read and understand them. With this experiment, we will examine the readability of loops written in different scripting languages and how easily developers can identify errors in code written in Bash, PowerShell, and Python.

7.1 Initial Considerations

To create an experiment that compares loops in scripting languages, we wanted to find a common use case of scripting languages that uses loops. We considered the task of reading lines from a file. As a typical use case for scripting, this process is even described in beginner learning books, such as Newham’s Learning the Bash Shell [24]. As we do with the previous two experiments, we again want to present the participant with code snippets. The participant should identify whether the code is valid or invalid. Again, we want to measure the reaction time for each task as an approximation for the reading time.

Code Snippets We want to present the participant with code snippets for several languages, each demonstrating a method for looping through a file. The idea behind each snippet is to use a single input file that contains a list of different filenames. The code should then loop over each of these filenames. Since there are many ways to loop through files in each scripting language, we have selected only a few methods for each one. The experiment focuses on nine different methods to loop through the input file—four methods for Bash, four methods for PowerShell, and one method for Python. The loop body is kept simple and is represented by code that outputs ". . .". Additionally, the variable used to refer to the input file is not assigned in the code and the participants are told to assume that this variable is already assigned. This removes unnecessary complexity so that the focus remains on the looping mechanism.

Error Detection Since the experiment’s scope is to study the difficulties of error detection in different scripting languages and looping methods, errors are intentionally introduced into the code snippets. We want the participant to read through the whole code snippet and not focus on certain parts only. Therefore, errors may be randomly introduced into every token of the code. Consistent with the previous experiment, we define an error as a

modification of the code that would cause an exception when running the code or alter the way the code behaves in an unexpected way. We want to introduce a unique error for every token. The errors should ideally mimic errors a developer could introduce when writing code, e.g. typos, missing semicolons or similar.

Task Preparation Switching between different languages and looping methods can take time. Therefore, before each task, participants should be shown the method used to loop through the file. This initial snippet should contain no errors. After reviewing the code, the participant will be free to proceed to the task, where the same snippet will be shown again, but it may now contain an error.

Task As previously stated, this experiment focuses on error detection. Therefore, the subject should respond, whether the presented code snippet is valid or invalid.

Further considerations can be found in Section 4.1.

7.2 Experiment Layout

The experiment layout consists of the following variables:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his result. This variable should approximate the reading time.
 - **correctness:** Whether the given response is correct or not.
- **Independent Variables:**
 - **method:** The looping method used for the task. For this variable there are nine treatments, the four methods used with Bash, the four methods used with PowerShell, and the method used with Python. The labels can be found in Figure 5.
 - **error:** Whether the code snippet contains an error or not.
- **Fixed Variables:**
 - **repetitions:** The experiment repeats each treatment combination 20 times for a total of 360 code snippets.
- **Task:** *“Press [1] if the code snippet is valid and [e] if the code snippet is invalid and contains an error.”*

Again, the goal of the experiment is to see if the independent variables influence the dependent variables. This experiment was conducted by one subject.

7.3 Task Design

b_map:

```
mapfile -t files < "$InputFile"
for file in "${files[@]}; do
    echo "..."
done
```

b_wc:

```
while read -r file || [[ -n "$file" ]]; do
    echo "..."
done < "$InputFile"
```

b_woc:

```
while read -r file; do
    echo "..."
done < "$InputFile"
```

b_cat:

```
for file in $(cat "$InputFile"); do
    echo "..."
done
```

pS_net:

```
$reader = [System.IO.StreamReader]::new($InputFile)
while (($line = $reader.ReadLine()) -ne $null) {
    Write-Output "..."
}
$reader.Close()
```

pS_fe:

```
foreach ($line in Get-Content $InputFile) {
    Write-Output "..."
}
```

pS:

```
Get-Content $InputFile | ForEach-Object {
    Write-Output "..."
}
```

pS_wo:

```
Get-Content $InputFile -ReadCount 1 | ForEach-Object {
    Write-Output "..."
}
```

py:

```
with open(InputFile, "r") as file:
    for line in file:
        print("...")
```

Figure 5: Code templates for the preliminary loops experiment

b_map:

```
mapfile -t files < $InputFile
for file in "${files[@]}"; do
    echo "..."
done
```

pS_fe:

```
foreach ($line Get-Content $InputFile) {
    Write-Output "..."
}
```

py:

```
with open($InputFile, "r") as file:
    for line in file:
        print("...")
```

Figure 6: Example code snippets containing an error for the loops preliminary experiment

The code snippets for each task are created from one of the nine templates (see Figure 5). Each template contains a different method to loop through an input file. These templates correspond to the code snippets shown when the snippet contains no error. For the correct code snippets, all tokens are presented exactly as shown in the figure with none being dynamically changed. If the snippet contains an error, a random token is selected and substituted with a predefined invalid token designed to mimic mistakes that a developer might introduce during coding. There are multiple predefined invalid tokens for every method. Figure 6 presents three example code snippets containing an error—for the `b_map` method, the `pS_fe` method, and the `py` method. The first code snippet (`b_map`) is invalid because the `$InputFile` is missing the quotation marks, which might cause unintended word splitting. The `pS_fe` snippet is invalid because it misses the `in` keyword between `$line` and `Get-Content`. The `py` code snippet is invalid because the `InputFile` is referenced with a leading `$`, which is not done in Python.

7.4 Analysis

Comparing the correctness of the answers given by the subject with a χ^2 -Test shows no significant effect—meaning that the looping method had no influence on whether the answer was correct or not (see Table 3). All nine methods resulted in some incorrect answers, with the Python method producing the most (8 incorrect answers out of 40 tasks) and the PowerShell looping method using `ForEach-Object` command without options producing the fewest (1 incorrect answer out of 40 tasks).

Table 3: Experiment results for the loops preliminary experiment

ANOVA on Reaction Time (approximator for Reading Time)								
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M	$\Delta M\%$ (pS)
method	8	57.53	< .001	.574	pS	1382; 1655	1519	0%
					b_woc	1609; 1998	1804	+18.8%
					pS_fe	1611; 2093	1852	+21.9%
					b_cat	1916; 2367	2141	+40.9%
					pS_wo	1904; 2377	2141	+40.9%
					py	2613; 3196	2904	+91.2%
					b_wc	2683; 3272	2977	+96.0%
					b_map	3003; 3831	3417	+125.0%
error	1	29.68	< .001	.08	pS_net	4553; 5646	5099	+235.7%
					no error	2715; 3119	2917	
method * error	8	1.32	.231	.03	error	2174; 2594	2384	
					pS/no error	1521; 1809	1665	
					pS/error	1146; 1599	1373	
					b_woc/no error	1804; 2145	1975	
					b_woc/error	1280; 1985	1633	
					pS_fe/no error	1948; 2420	2184	
					pS_fe/error	1134; 1907	1520	
					b_cat/no error	2157; 2607	2382	
					b_cat/error	1519; 2283	1901	
					pS_wo/no error	2137; 2644	2390	
					pS_wo/error	1500; 2282	1891	
					py/no error	2619; 3175	2897	
					py/error	2368; 3456	2912	
					b_wc/no error	2917; 3616	3267	
					b_wc/error	2219; 3157	2688	
					b_map/no error	3249; 4340	3795	
					b_map/error	2414; 3664	3039	
					pS_net/no error	5104; 6294	5699	
					pS_net/error	3609; 5390	4500	

χ^2 -Test on method						
Variable	df	χ^2	p	N	TRT	Incorrect Answers
method	8	8.16	.418	40	b_map	6
				40	b_wc	4
				40	b_woc	4
				40	b_cat	4
				40	pS_net	6
				40	pS_fe	3
				40	pS	1
				40	pS_wo	4
				40	py	8

ANOVA on Reaction Time (approximator for Reading Time) with language							
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M
language	2	.835	.435	.005	Bash	2410; 2759	2585
					PowerShell	2378; 2927	2653
					Python	2613; 3196	2904
error	1	4.803	.029	.013	see above		
language * error	2	.943	.390	.005	Bash/no error	2625; 3084	2854
					Bash/error	2062; 2569	2315
					PowerShell/no error	2593; 3376	2985
					PowerShell/error	1942; 2700	2321
					Python/no error	2619; 3175	2897
					Python/error	2368; 3456	2912

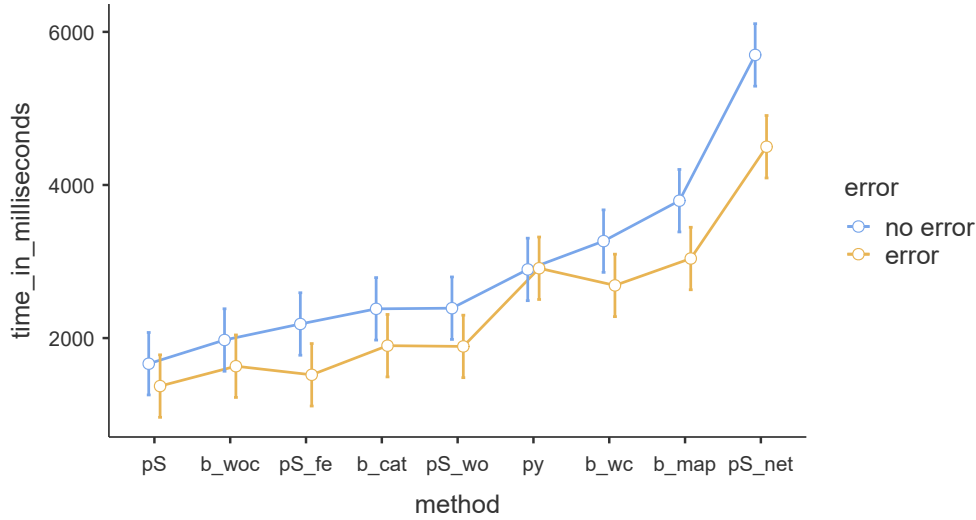


Figure 7: Interaction diagram method * error

While the correctness was not significantly influenced by the looping method, the reaction time was significantly affected by it, as the ANOVA in Table 3 shows ($p < .001$ for the method variable). The method variable also has a very large effect with a η_p^2 of .578. We also found a statistically significant effect for the error variable ($p < .001$), which had a medium effect size with a η_p^2 of .08. However no interaction effect between looping method and error was found ($p = .231$).

The mean reaction time for the slowest looping method—the PowerShell method using the `System.IO.StreamReader` class from the .Net framework—was 235.7% longer than that of the fastest method—the PowerShell method using the `ForEach-Object` command without options. This means that both the smallest and largest means were from methods using PowerShell, showing significant fluctuations within the same scripting language. Comparing the means of the fastest looping methods for each language, Python was the slowest (2904 milliseconds), Bash the second fastest (1804 milliseconds)—using a while loop with the `read` command—and PowerShell the fastest (1519 milliseconds) with the aforementioned method. Figure 7 visualizes the times for each method and shows the interaction between the method and the error (labels can be seen in Table 5).

When comparing the programming languages instead of the looping method, we found no significant effect on mean times ($p = .435$). The means were relatively close together, with Python—the slowest language—showing reaction times only 12.3% longer than Bash, the fastest language. As we found no significant effect, a repeated experiment with other participants or a larger set of code snippets could yield different results, and the relative ranking of

language reaction times might change.

7.5 Discussion

The results of the experiment revealed a large effect of the looping method used on measured reaction times. However, results indicate that while the looping method was significant, the language used was not. Examining the methods used in the experiment, not all behave similarly. For instance, when looping through an input file containing a list of file names, the looping method using the `cat` command in Bash (see `b_cat` in Figure 5) may fail if file names contain whitespaces, as this leads to word splitting and incorrect file recognition. Additionally, the `b_woc` (see Figure 5) does not iterate over the last item in the input file, making the condition used in `b_wc` (see Figure 5) necessary. Furthermore, the `-ReadCount` option in the `pS_wo` code snippet is not needed for that use case, as the default value for `-ReadCount` already is 1. Meaning for a follow-up experiment, these looping methods might be excluded.

The experiment was designed so that participants first read and memorized the correct code snippet, and then, in a next step, read the code snippet again—which might now contain an error—and completed the task. In the current experiment, only the times for the second reading and task completion were measured. It may also be insightful to measure the time spent on the initial reading and memorization of the code snippet.

Additionally, similar to the discussion about the conditional preliminary experiment in Section 6.6, this experiment may also be affected by order effects. Again, multiple consecutive tasks may use the same looping method, which could lead to carryover effects, fatigue, or boredom due to repetition. Varying the order of tasks could reduce those effects, if present.

8 Conditionals Main Experiment

Following two preliminary experiments on conditional statements in scripting languages, we designed and conducted the main experiment focusing on conditionals. This chapter describes the initial considerations leading to the experiment, as well as its design, conduction, and analysis of results.

8.1 Initial Considerations

The findings of the preliminary experiments indicate that the scripting language affects the readability of conditional statements. The experiment design for this experiment is closely related to the preliminary experiment in Section 6, with a few adjustments, which are detailed here.

Code Snippets In the preliminary experiment the body of the conditional statement was meant to be ignored. Therefore, the body contained code that outputs ". . ." in the respective language. To further reduce complexity and ensure that the conditional body is ignored, this is replaced by a single line comment in this experiment.

Error Detection For this experiment, after each task, if the code snippet contained an error, the invalid token is highlighted and the error explained. The explanation details how the invalid token leads to an error. This may help participants during the training phase and the main experiment.

Participants Because participants do not have to write code and, given the limited complexity of the tasks and code snippets, we expect that participants with basic programming knowledge will be able to solve the tasks reliably. For the experiment, we recruited undergraduate students, professionals working in programming-related fields, and trainees in IT-related fields.

N-of-1 Similar to the preliminary experiments, this experiment was designed as an N-of-1 trial. N-of-1 trials give all participants all treatment combinations. This allows to study the effect on all participant—by using repeated N-of-1 studies on the participants—and on single participants.

Order Effects As we mentioned in Section 2.2 and discussed about the preliminary experiment in Section 6.6, within-subject designs and N-of-1 trials are susceptible to order effects. To mitigate that, we decided to not have two consecutive code snippets use the same treatment. While the code snippets are randomly ordered, we make sure that treatments alternate. This reduces order effects, minimizes carryover effects, and aims to reduce boredom and fatigue from repetition.

8.2 Experiment Layout

The experiment layout consists of the following variables:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his result. This variable should approximate the reading time.
 - **correctness:** Whether the given response is correct or not.
- **Independent Variables:**
 - **language:** The programming language used for the task. For this variable there are three treatments, either Bash, PowerShell, or Python.
 - **error:** Whether the conditional statement contains an error or not.
- **Non-Controlled Variables:**
 - **background:** student (s), professional (p), or trainee (t)
- **Fixed Variables:**
 - **code snippets:** Initially randomly generated seeded code snippets. Each participant received the same code snippets
 - **repetitions:** The experiment repeats each treatment combination 8 times for a total of 48 code snippets.
 - **order:** The order of the code snippets was the same for all participants
- **Task:** *“Press [1] if the code has no error and [e] if the code has an error.”*

The goal of the experiment is to see if the independent variables influence the dependent variables, i.e. if the language and error influence the reaction time and correctness.

8.3 Task Design

The task design is similar to the task design for the preliminary experiment on conditionals in Section 6.3 with some changes to the templates condition body. Figure 8 shows the updated templates used for this experiment. Figure 9 presents an example code snippet for each language containing an error. The Bash snippet is invalid because it omits a whitespace at the end of the condition, between `18` and `]]`. The PowerShell code is invalid because it is missing the `-` in the `-eq` comparison operator. Finally, the Python snippet is invalid because it uses the wrong comparison operator (`=>` instead of `>=`).

Bash:

```
if [[ "$name" == "Max" && $age -ge 18 ]]; then
    # Condition body
fi
```

PowerShell:

```
if ($name -eq "Max" -and $age -ge 18) {
    # Condition body
}
```

Python:

```
if name == "Max" and age >= 18:
    # Condition body
```

Figure 8: Code templates for the main conditionals experiment

Bash:

```
if [[ "$name" == "Max" && $age -ge 18]]; then
    # Condition body
fi
```

PowerShell:

```
if ($name eq "Max" -and $age -ge 18) {
    # Condition body
}
```

Python:

```
if name == "Max" and age => 18:
    # Condition body
```

Figure 9: Example code snippets containing an error for each language

8.4 Measurement and Conduction

The measurement was done similarly to the preliminary experiment about conditionals (see 6.4). However, since this experiment was conducted by multiple participants some things changed.

This experiment was not executed by the researcher, but by voluntary participants. Using the interface [3], the participants completed the experiment without supervision. After the experiment, participants were asked to send the generated csv-file to the researcher.

The experiment was executed on 10 volunteers—5 undergraduate students, 4 professionals, and 1 trainee in an IT-related field. One undergraduate student was removed from the study, as they answered [1] to every question, resulting in a total of 9 participants. The participants all possess general programming knowledge, while some possess more advanced knowledge as others.

8.5 Analysis

The χ^2 -Test in Table 4 reveals that the scripting language used had no significant effect on whether the response given by the participants was correct ($p = .642$). Out of the 432 tasks, 77 were answered incorrectly, resulting in an overall error rate of 17.8%. The error rates by programming language were 17.4% for Bash, 16.0% for PowerShell, and 20.1% for Python. The number of incorrect answers was also analyzed per participant and is presented in Table 5. Although the total number of errors varied considerably between participants—participant 2 made only one error, whereas participant 3 made 18 errors—no participant showed a significant effect of the scripting language on the number of incorrect answers.

The data on the reaction time was analyzed using an ANOVA, for which data outliers outside of $[Q1 - 1.5IQR, Q3 + 1.5IQR]$ were removed. This removes 35 of 432 data points (8.1%). Both independent variables—language and error—were significant on the reaction time (see $p < .001$ in Table 4) with a medium effect ($\eta_p^2 = .095$ and $\eta_p^2 = .066$). Python showed the shortest mean reaction time at 4359 milliseconds. PowerShell's mean reaction time was 5653 ms, representing a 29.7% increase relative to Python. Bash was the slowest, with a mean reaction time of 6771 ms, which is 55.3% larger than Python's.

A follow-up post hoc test comparing PowerShell and Python confirmed a significance between those languages, contrary to the result from the preliminary experiment (see Table 2).

The ANOVA also shows an interaction effect of the language and background ($p < .001$) with a medium effect ($\eta_p^2 = .066$). The visualization in Figure 10 shows that language had a larger effect on professionals and trainees than on students.

Further, the reaction times were analyzed separately for each participant. To address potential outliers, three analyses were conducted. Table 6 shows the results without excluding outliers. Table 7 shows the results after excluding global outliers, consistent with the prior ANOVA on the complete data set. Finally, Table 8 shows the results after removing local outliers, defined as values outside $[Q1 - 1.5IQR, Q3 + 1.5IQR]$ for each participant.

Without removing outliers, four of the nine participants revealed a significant effect of the language variable (see participants 2, 4, 5, and 8 in Table 6), one participant approaches significance (see participant 7 with $p = .06$), and the remaining four showed no significant effect (participants 1, 3, 6, and 9). For some participants, the 95% confidence intervals were very wide, with participant 7 having an upper bound of 109465 ms and a mean reaction time of 52717 ms for Bash. Participant 6 and 9 also stood out from the other participants, with p values of .983 and .979 respectively. Participant 6 had the lowest means of ~3.5

seconds across languages, while participant 9 showed the largest means—apart from Bash for participant 7—with around 14 seconds across languages. Further comparing the mean times for each language reveals that Bash was the slowest language and Python was the fastest for all participants.

When global outliers were removed (see Table 7), some results changed. Participant 3, showed a significant effect of language after a single Bash outlier was excluded ($p = .027$). The results for participants 7 and 9 also changed substantially. However, these findings may be unreliable given the extend of data removed. For participant 7, five out of 16 Bash tasks were excluded, making it almost a third of all the tasks. For participant 9, eleven Python, six PowerShell, and three Bash tasks were excluded. For every participant except participant 3, Bash remained the slowest language, while Python remained the fastest. For participant 3, however, removing the outlier for Bash made PowerShell the slowest language, with Python still the fastest.

Therefore, a third analysis removes only local outliers to see whether the observed effects persist under less extensive data exclusion. This analysis shows, that while participant 1 was only approaching significance with the first two analyses, the language variable is significant with 8 outliers removed ($p = .003$, removing 16.7% of all tasks). Additionally, while the first analysis revealed an approaching significance for participant 7, this analysis shows a significant effect of the scripting language ($p = .034$, removing two outliers for each language). This analysis also confirms no significance of the language for participant 9 ($p = .830$).

As in the second analysis, removing outliers changed the ranking of the fastest languages for some participants. For participant 2, the ranking remained similar to the second analysis. For participant 6, excluding two outliers made PowerShell the slowest language, while Python remained the fastest. Removing four outliers for participant 9 made PowerShell the fastest language and Python the slowest.

In all three analyses, for participants that showed language to be significant, the effect was rather large, with participant 4 having the largest measured effect ($\eta_p^2 = .374$ in Table 8) and participant 2 having the smallest effect ($\eta_p^2 = .154$).

8.6 Threats to Validity

Sample size and outliers Both the sample size of participants and the amount of tasks was small—nine participants and 48 tasks per participant with repeating each treatment combination eight times. Due to this, the results of the experiment are highly susceptible to outliers, as confirmed by the analysis. Without removing outliers, the mean reaction time of participant 7 with Bash (see $M = 52717$ ms in Table 6) was more than 14 times

Table 4: Results for the main experiment on conditionals

ANOVA on Reaction Time (approximator for Reading Time)								
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M	$\Delta M\%$ (Python)
language (l)	2	19.94	< .001	.095	Bash	6140; 7403	6771	+55.3%
					PowerShell	5170; 6136	5653	+29.7%
					Python	4009; 4709	4359	0%
background (b)	2	47.69	< .001	.201	p	5151; 6092	5622	
					t	8996; 12035	10516	
					s	4580; 5155	4867	
error	1	26.71	< .001	.066	no error	5699; 6637	6168	
					error	4690; 5451	5071	
l * b	4	2.84	< .001	.066	Bash/p	5957; 8013	6985	
					Bash/t	10354; 14649	12501	
					Bash/s	4892; 5900	5396	
					PowerShell/p	4823; 6281	5552	
					PowerShell/t	7039; 12292	9665	
					PowerShell/s	4583; 5658	5121	
					Python/p	3856; 4976	4416	
					Python/t	4045; 10061	7053	
					Python/s	3680; 4508	4094	
l * error	2	1.21	.300	.006	<i>omitted due to non-significant effect</i>			
b * error	2	5.91	.003	.030	p/no error	5744; 7195	6470	
					p/error	4255; 5403	4829	
					t/no error	11236; 14635	12935	
					t/error	6939; 10961	8950	
					s/no error	4685; 5559	5122	
					s/error	4235; 4986	4610	
l * b * error	4	1.06	.378	.011	<i>omitted due to non-significant effect</i>			

Post Hoc Test			
Variable	Variable	df	p
Bash	PowerShell	379	.010
	Python	379	< .001
PowerShell	Python	379	.001

χ^2 -Test on language						
Variable	df	χ^2	p	N	TRT	Incorrect Answers
language	2	.885	.642	144	Bash	25
				144	PowerShell	23
				144	Python	29

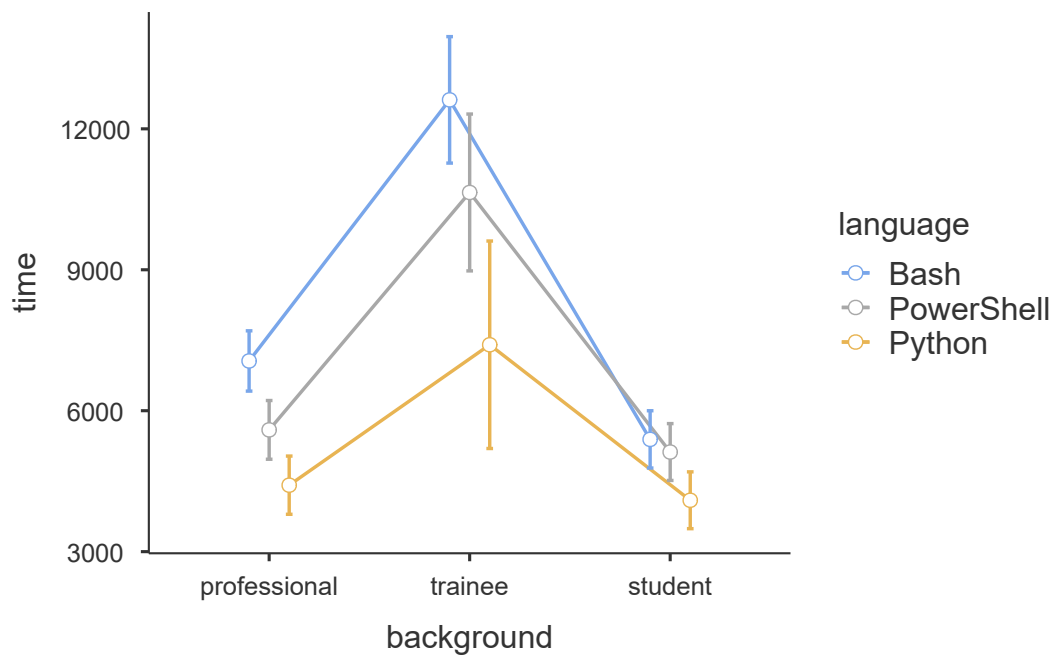


Figure 10: Interaction diagram background * language

Table 5: Incorrect answers of participants per scripting language

χ^2 -Test on language per participant							
participant	background	df	χ^2	p	N	TRT	Incorrect Answers
1	p	2	3.28	.194	16	Bash	1
					16	PowerShell	4
					16	Python	5
2	p	2	2.04	.360	16	Bash	1
					16	PowerShell	0
					16	Python	0
3	s	2	.533	.766	16	Bash	7
					16	PowerShell	6
					16	Python	5
4	s	2	1.200	.549	16	Bash	2
					16	PowerShell	4
					16	Python	2
5	p	2	2.67	.264	16	Bash	2
					16	PowerShell	4
					16	Python	6
6	s	2	1.200	.549	16	Bash	4
					16	PowerShell	2
					16	Python	2
7	p	2	3.13	.210	16	Bash	2
					16	PowerShell	0
					16	Python	3
8	s	2	.667	.717	16	Bash	4
					16	PowerShell	3
					16	Python	5
9	t	2	2.13	.344	16	Bash	2
					16	PowerShell	0
					16	Python	1

Table 6: Results per participant for Bash, PowerShell, and Python (without removing outliers)

ANOVA on Reaction Time of language							
participant	background	p	η_p^2	N	TRT	$CI_{95\%}$	M
1	p	.195	.070	16	Bash	3923; 9477	6700
				16	PowerShell	2154; 8645	5400
				16	Python	2281; 4855	3568
2	p	.012	.177	16	Bash	6978; 13147	10062
				16	PowerShell	5629; 9596	7613
				16	Python	4152; 6716	5434
3	s	.115	.091	16	Bash	5317; 10538	7927
				16	PowerShell	6269; 8827	7548
				16	Python	4840; 6430	5635
4	s	< .001	.362	16	Bash	4120; 6051	5085
				16	PowerShell	3987; 4808	4398
				16	Python	2578; 3349	2963
5	p	.035	.138	16	Bash	5722; 9187	7454
				16	PowerShell	4837; 7816	6326
				16	Python	3374; 6091	4732
6	s	.983	.001	16	Bash	3059; 4295	3677
				16	PowerShell	3057; 4173	3615
				16	Python	2676; 4510	3593
7	p	.060	.118	16	Bash	-4031; 109465	52717
				16	PowerShell	3645; 9596	6621
				16	Python	3248; 9408	6328
8	s	.004	.215	16	Bash	5187; 7061	6124
				16	PowerShell	4045; 5800	4922
				16	Python	3501; 4867	4184
9	t	.979	.001	16	Bash	11604; 17051	14327
				16	PowerShell	10180; 18019	14099
				16	Python	10775; 16977	13876

Table 7: Results per participant for Bash, PowerShell, and Python (removing global outliers)

ANOVA on Reaction Time of language								
participant	background	p	η_p^2	N	TRT	$CI_{95\%}$	M	outliers
1	p	.091	.105	15	Bash	3705; 7795	5750	1
				15	PowerShell	2738; 5214	3976	1
				16	Python	2281; 4855	3568	0
2	p	.019	.169	15	Bash	6578; 11714	9146	1
				15	PowerShell	5336; 8805	7071	1
				16	Python	4152; 6716	5434	0
3	s	.027	.151	15	Bash	5776; 7793	6784	1
				16	PowerShell	6269; 8827	7548	0
				16	Python	4840; 6430	5635	0
4	s	< .001	.362	16	Bash	4120; 6051	5085	0
				16	PowerShell	3987; 4808	4398	0
				16	Python	2578; 3349	2963	0
5	p	.007	.203	16	Bash	5722; 9187	7454	0
				16	PowerShell	4837; 7816	6326	0
				15	Python	3284; 5229	4257	1
6	s	.983	.001	16	Bash	3059; 4295	3677	0
				16	PowerShell	3057; 4173	3615	0
				16	Python	2676; 4510	3593	0
7	p	.729	.017	11	Bash	3497; 6582	5040	5
				14	PowerShell	3550; 5910	4730	2
				14	Python	3372; 5410	4391	2
8	s	.004	.215	16	Bash	5187; 7061	6124	0
				16	PowerShell	4045; 5800	4922	0
				16	Python	3501; 4867	4184	0
9	t	.015	.286	13	Bash	10354; 14649	12501	3
				10	PowerShell	7039; 12292	9665	6
				5	Python	4045; 10061	7053	11

Table 8: Results per participant for Bash, PowerShell, and Python (removing outliers per participant)

participant	background	ANOVA on Reaction Time of language						
		p	η_p^2	N	TRT	$CI_{95\%}$	M	outliers
1	p	.003	.274	13	Bash	3468; 5494	4481	3
				13	PowerShell	2608; 3795	3202	3
				14	Python	2242; 3293	2767	2
2	p	.025	.154	15	Bash	6578; 11714	9146	1
				16	PowerShell	5629; 9596	7613	0
				16	Python	4152; 6716	5434	0
3	s	.020	.166	14	Bash	5642; 7330	6486	2
				16	PowerShell	6269; 8827	7548	0
				16	Python	4840; 6430	5635	0
4	s	< .001	.374	16	Bash	4120; 6051	5085	0
				15	PowerShell	3955; 4558	4257	1
				16	Python	2578; 3349	2963	0
5	p	.007	.203	16	Bash	5722; 9187	7454	0
				16	PowerShell	4837; 7816	6326	0
				15	Python	3284; 5229	4257	1
6	s	.665	.019	15	Bash	3030; 3878	3454	1
				16	PowerShell	3057; 4173	3615	0
				15	Python	2605; 3955	3280	1
7	p	.034	.159	14	Bash	3260; 27831	15546	2
				14	PowerShell	3550; 5910	4730	2
				14	Python	3372; 5410	4391	2
8	s	.002	.246	16	Bash	5187; 7061	6124	0
				15	PowerShell	3910; 5474	4692	1
				16	Python	3501; 4867	4184	0
9	t	.830	.009	13	Bash	12197; 15461	13829	3
				15	PowerShell	9730; 16017	12874	1
				16	Python	10775; 16977	13876	0

larger than the mean reaction time of participant 6 ($M = 3677$ ms) with the same language. Without accommodating for outliers, differences this large have a large effect of the analysis, especially if the total sample size is small. Therefore, for the analysis, we removed outliers outside of the $[Q1 - 1.5IQR, Q3 + 1.5IQR]$ interval. When considering global outliers, a substantial portion of data was excluded for some participants (see Table 7). For instance, 41.7% of data points were removed for participant 9. When considering just local outliers (see Table 8), this approach still excluded up to 16.7% of data points per participant (see participant 1).

Participants The participants vary in their programming knowledge and especially in their knowledge about scripting languages. The selection process did not account for differences in scripting knowledge either. Some participants reported that they had no prior experience with certain languages included in the experiment.

Code Snippets The code snippets used for the experiment were short and only included two comparisons and one logical and operator to combine them. Conditional statements in any of the three scripting languages can drastically increase in complexity, which could impact the readability.

8.7 Discussion

The experiment results indicate, that the scripting language used influences the readability of conditional statements. After excluding outliers, Python had the lowest mean reaction time, with PowerShell and Bash showing mean times that were 29.7% and 55.3% higher, respectively (see Table 4). The language significantly influenced reaction times with a medium effect size ($p < .001$, $\eta_p^2 = .095$).

Analyses at the individual level generally supported the overall results. Three approaches were applied for each participant: including all data, excluding global outliers, and excluding local outliers for each participant (see Tables 6, 7, and 8). In some cases, excluding global outliers resulted in excessive data removal; for instance, for participant 9, 11 out of 16 Python tasks (68.75%) were eliminated. Therefore, the third approach, excluding only local outliers, was employed. The analyses indicated that outliers substantially affected the results for some participants. For instance, participant 1 showed no significant effect of language when all data points were included ($p = .195$, $\eta_p^2 = .070$), but exhibited a significant effect with a very large effect size after excluding local outliers ($p = .003$, $\eta_p^2 = .274$). For participant 6, the effect of the language was not observable in any of the approaches ($p = .983$, $p = .983$, and $p = .665$) and for others it was only observable in some approaches. When all data was included, 4 of 9 participants showed a significant effect of language (see Table 6). Excluding global outliers increased this number to 6 (see Table 7), and excluding local outliers further increased it to 7 (see Table 8). We think that increasing the number of tasks per participant would reduce the influence of outliers on the results and increase the number of participants showing significant effect.

Besides readability, the correctness of answers given was also measured. Analyses of both the complete data set (see $p = .642$ in Table 4) and individual participants indicated no significant effect of language on error rates (see Table 5).

As described in Section 8.6, this experiment has some possible flaws, mainly resulting from the small number of tasks and choice of participants. A follow-up experiment with a larger sample size of participants could determine whether the results are generalizable. Additionally, the conditional statements can be modified to cover different aspects of scripting, like wildcards or piping, to see if the observed behavior persists.

9 Loops Main Experiment

Following the preliminary experiment on loops, the main experiment was designed and conducted. Similar to the conditionals main experiment in Section 8, we again focus on a control flow structure in scripting languages, only now: loops.

9.1 Initial Considerations

The results of the preliminary experiment on loops in Section 7 showed a large effect of the looping method on reaction times, while the discussion in Section 7.5 highlighted several points for improvement. This experiment's design closely resembles the preliminary experiment, therefore, only the differences are highlighted here.

Code Snippets As in the previous main experiment, the code producing the `" . . . "` output is removed. Instead, a single line comment is added to represents the body of the loop. Participants are informed that this line could not generate an error, to ensure that it is ignored during the experiment. The code snippets contain different methods to loop through an input file. The six methods used are taken from the preliminary loops experiment, while three methods are excluded, following the discussion in Section 7.5. The remaining looping methods consists of two methods in Bash, three methods in PowerShell, and one method in Python. Again, these are not the only ways to loop through an input file but these are the ones we focus on in this experiment.

Error Detection Similar to the main experiment on conditionals, if the code snippet contains an error, the invalid token is highlighted and the error explained, which may assist participants during both the training phase and the main experiment.

Task Preparation Again, participants are shown the method used to loop through the file before each task, referred to as the pretask. However, for this experiment, the time it takes to review the code will be measured.

Participants This experiment was executed on the same participants as the conditionals main experiment.

N-of-1 As in the main experiment on conditionals, this study uses repeated N-of-1 trials. All participants receive all treatment combinations, allowing analysis of treatment effects on all participants as well as on single participants.

Order Effects To mitigate order effects, we assured that no two consecutive code snippets use the same treatment. While they might still use the same programming language, the looping methods alternate. This aims to reduce order effects, minimize carryover effects, and reduce boredom and fatigue from repetition.

9.2 Experiment Layout

The experiment layout consists of the following variables:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his result. This variable should approximate the reading time.
 - **pretask reading time:** The time it takes the participant to review the initial code snippet.
 - **correctness:** Whether the given response is correct or not.
- **Independent Variables:**
 - **method:** The looping method used for the task. For this variable there are nine treatments, the four methods used with Bash, the four methods used with PowerShell, and the method used with Python.
 - **error:** Whether the conditional statement contains an error or not.
- **Non-Controlled Variables:**
 - **background:** student (s), professional (p), or trainee (t)
- **Fixed Variables:**
 - **code snippets:** Initially randomly generated seeded code snippets. Each participant received the same code snippets
 - **repetitions:** The experiment repeats each treatment combination 5 times for a total of 60 code snippets.
 - **order:** The order of the code snippets was the same for all participants
- **Task:** *“Press [1] if the code has no error and [e] if the code has an error.”*

The goal of the experiment is to see if the independent variables influence the dependent variables, i.e. if the method and error influence the reaction time and correctness.

bmap

```
mapfile -t files < "$input"
for file in "${files[@]}; do
    # loop body
done
```

bwhile

```
while read -r file || [[ -n "$file" ]]; do
    # loop body
done < "$input"
```

pS

```
Get-Content $input | ForEach-Object {
    # loop body
}
```

pSforeach

```
foreach ($line in Get-Content $input) {
    # loop body
}
```

pSreader

```
$reader = [System.IO.StreamReader]::new($input)
while (($line = $reader.ReadLine()) -ne $null) {
    # loop body
}
$reader.Close()
```

pY

```
with open(input, "r") as file:
    for line in file:
        # loop body
```

Figure 11: Code templates for the main loops experiment

9.3 Task Design

The task design for this experiment is similar to the design of the preliminary experiment on loops (see Section 7.3). However, some looping methods and the corresponding templates have been removed, whereas the remaining have been slightly updated. The updated templates used for this experiment are shown in Figure 11.

Figure 12 presents some example invalid code snippets for three methods. In the first example, using the `bwhile` method, the wrong option is used for the `read` command. The correct option for the command would be `-r`. The `pS` code snippet is invalid as it uses `||` instead of `|`. Lastly, in the `pSreader` code example, the first `$reader` is missing its leading `$`.

9.4 Measurement and Conduction

The measurement and conduction of the experiment was done similarly to that of the conditionals main experiment (see Section 8.4). The experiment was ex-

bwhile

```
while read -n file || [[ -n "$file" ]]; do
  # loop body
done < "$input"
```

pS

```
Get-Content $input || ForEach-Object {
  # loop body
}
```

pSreader

```
reader = [System.IO.StreamReader]::new($input)
while (($line = $reader.ReadLine()) -ne $null) {
  # loop body
}
$reader.Close()
```

Figure 12: Example code snippets containing an error for the main loops experiment

ecuted on the same participants with a similar interface [3]. Participants completed the experiment without supervision and sent the generated csv-files to the researcher.

Similar to the previous experiment, one student was removed from the study, as they gave the same answer to every task with unrealistically short reaction times.

9.5 Analysis

The number of incorrect responses was analyzed using a χ^2 -Test. In Table 9 we see that the the looping method used was significant on the correctness of the given responses ($p = .002$). For each looping method, 90 tasks were answered. The `pS` and `pSforeach` code snippets (see code snippets in Figure 11) resulted the fewest incorrect answers (12 and 13, respectively), whereas the `bmap` and `pSreader` code snippets yielded the most (28 and 30, respectively). Consequently, PowerShell included the looping methods with both the lowest and highest error counts. The error counts were also analyzed for each participant separately and are shown in Table 11. This shows that, while we measured an overall significance of the looping method on the correctness for the whole data set, only one participant confirmed this effect ($p = .047$ for participant 8). However, two participants—participant 5 and participant 6—approached significance ($p = .061$ and $p = .055$). For participant 8, the `py` and `pSreader` code snippets yielded the most errors, with error rates of 60% and 50%. Participant 6 also struggled with with the `pSreader` code with an error rate of 60%. Contrary to this, the `bwhile` and `bmap` code snippets caused the most invalid answers for participant 5, with error rates of 60% and 50%.

Reaction time data was analyzed using an ANOVA. As in the previous exper-

Table 9: Results of loops experiment on reaction time

ANOVA on Reaction Time (approximator for Reading Time)								
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M	$\Delta M\%$ (pS)
method	5	31.14	< .001	.241	pS	3106; 3882	3494	0%
					pSforeach	3536; 4598	4067	+16.4%
					py	4615; 5838	5227	+49.6%
					bwhile	6266; 8192	7229	+106.9%
					bmap	7321; 9586	8453	+141.9%
					pSreader	7867; 9653	8760	+150.7%
error	1	5.91	.015	.012	no error	6122; 7200	6661	
					error	5325; 6340	5833	
method * error	5	2.36	.039	.024	pS/no error	3025; 4127	3576	
					pS/error	2849; 3984	3417	
					pSforeach/no error	3711; 5327	4519	
					pSforeach/error	2965; 4389	3677	
					py/no error	4537; 6296	5417	
					py/error	4145; 5910	5027	
					bwhile/no error	6847; 9795	8321	
					bwhile/error	4942; 7382	6162	
					bmap/no error	7864; 11271	9567	
					bmap/error	5872; 8857	7365	
					pSreader/no error	7201; 9288	8244	
pSreader/error	7782; 10653	9218						
χ^2 -Test on method								
Variable	df	χ^2	p	N	TRT	Incorrect Answers		
method	5	18.4	.002	90	pS	12		
				90	pSforeach	13		
				90	py	18		
				90	bwhile	18		
				90	bmap	28		
				90	pSreader	30		

iment, outliers outside of $[Q1 - 1.5IQR, Q3 - 1.5IQR]$ were removed, excluding 38 of 540 data points (7.04%). The analysis in Table 9 reveals a significant effect of the main factors method ($p < .001$) and error ($p = .015$), and a significance of the interaction method * error ($p = .039$). The looping method showed a very large effect size with ($\eta_p^2 = .241$). The pS code snippets had the lowest mean reaction time ($M = 3494$ ms), whereas pSreader had the highest ($M = 8760$ ms), representing a 150.7% increase over the former. The mean reaction time for the Python code snippet (py) was 49.6% higher than that of the pS snippet. The Bash code snippets (bwhile and bmap) showed even larger increases, with mean reaction times 106.9% and 141.9% higher than pS, respectively.

The effect size for the error was small ($\eta_p^2 = .012$). Tasks without an error ($M = 6661$ ms) took 14.2% longer to complete than tasks with an error ($M = 5833$ ms). However, there is also a small interaction effect of method and error ($\eta_p^2 = .024$). This interaction effect is illustrated in Figure 13. For the pSreader method, unlike the other methods, tasks containing an error took longer to complete than tasks without an error.

When grouping the looping methods by programming language and analyzing the reaction time data again (see Table 10), the programming language had

Table 10: Results of language on reaction time

ANOVA on Reaction Time (approximator for Reading Time) with language								
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M	$\Delta M\%$ (Python)
language	2	20.92	< .001	.078	Bash	7094; 8574	7834	+49.9%
					PowerShell	4989; 5934	5462	+4.5%
					Python	4615; 5838	5227	0%
error	1	4.61	.032	.009	see above			
language * error	2	3.88	.021	.015	Bash	7827; 10046	8937	
					Bash	5806; 7707	6757	
					PowerShell	4860; 6049	5454	
					PowerShell	4741; 6196	5468	
					Python	4537; 6296	5417	
					Python	4145; 5910	5027	
χ^2 -Test on language								
Variable	df	χ^2	p	N	TRT	Incorrect Answers		
language	2	1.95	.377	180	Bash	46		
				270	PowerShell	55		
				90	Python	18		

a significant effect ($p < .001$) with a medium effect size ($\eta_p^2 = .078$). Bash tasks had the longest mean reaction time ($M = 7834$ ms), whereas Python tasks were the fastest ($M = 5227$ ms). PowerShell had a mean reaction time 4.5% ($M = 5462$) longer than Python, while Bash's mean reaction time was 49.9% longer. However, the language had no effect on the correctness of the answers ($p = .377$).

Additionally, separate analyses were conducted for each participant. We analyzed the data both without removing outliers (see Table 12) and with outliers outside $[Q1 - 1.5IQR, Q3 + 1.5IQR]$ removed for each participant (see Tables 13). In the first analysis, all participant except participant 7 showed a significant effect of the method used ($p = .006$ for participant 1; $p < .001$ for all others). The effect sizes were very large, ranging from $\eta_p^2 = .258$ (participant 1) to $\eta_p^2 = .603$ (participant 6). For participants 1, 3, and 9, `bmap` was the slowest looping method, whereas the `pSreader` method resulted in the highest mean reaction times for the remaining participants. For participants 4 and 6, the `pSforeach` method was the fastest, for participant 7 `py` was the fastest, and for the remaining participants, `pS` method yielded the lowest mean reaction times.

In the second analysis (see Table 13), participant 7 also showed a significant effect of the method ($p = .017$), after four outliers were removed. For all participants, either `bmap` or `pSreader` remained the slowest looping method. The `pS` method resulted in the lowest mean reaction times for all except participants 4, 6, and 7, where `pSforeach` is the fastest.

Table 14 shows the reaction time analysis grouped by scripting language for each participant. Whereas the language analysis on the full data set revealed a significant effect of the language on the reaction time (see $p < .001$ in Table 9), the individual analyses confirmed this only for three participants: 1 ($p = .004$), 3 ($p < .001$), and 9 ($p = .003$). Bash was the slowest language for all 3 participants. For participants 3 and 9, Python was the fastest, whereas for participant

Table 11: Incorrect answers of participants per looping method

χ^2 -Test on method per participant					TRT	Incorrect Answers
participant	df	χ^2	p	N		
1	5	.410	.995	10	pS	3
				10	pSforeach	3
				10	py	3
				10	bwhile	2
				10	bmap	3
				10	pSreader	3
2	5	4.44	.487	10	pS	0
				10	pSforeach	2
				10	py	0
				10	bwhile	1
				10	bmap	2
				10	pSreader	1
3	5	3.75	.586	10	pS	1
				10	pSforeach	1
				10	py	1
				10	bwhile	3
				10	bmap	3
				10	pSreader	3
4	5	6.25	.283	10	pS	1
				10	pSforeach	0
				10	py	2
				10	bwhile	2
				10	bmap	4
				10	pSreader	3
5	5	10.6	.061	10	pS	2
				10	pSforeach	3
				10	py	0
				10	bwhile	6
				10	bmap	5
				10	pSreader	3
6	5	10.8	.055	10	pS	1
				10	pSforeach	1
				10	py	2
				10	bwhile	1
				10	bmap	3
				10	pSreader	6
7	5	2.93	.710	10	pS	2
				10	pSforeach	2
				10	py	3
				10	bwhile	1
				10	bmap	3
				10	pSreader	4
8	5	11.2	.047	10	pS	2
				10	pSforeach	1
				10	py	6
				10	bwhile	1
				10	bmap	2
				10	pSreader	5
9	5	6.63	.250	10	pS	0
				10	pSforeach	0
				10	py	1
				10	bwhile	1
				10	bmap	3
				10	pSreader	2

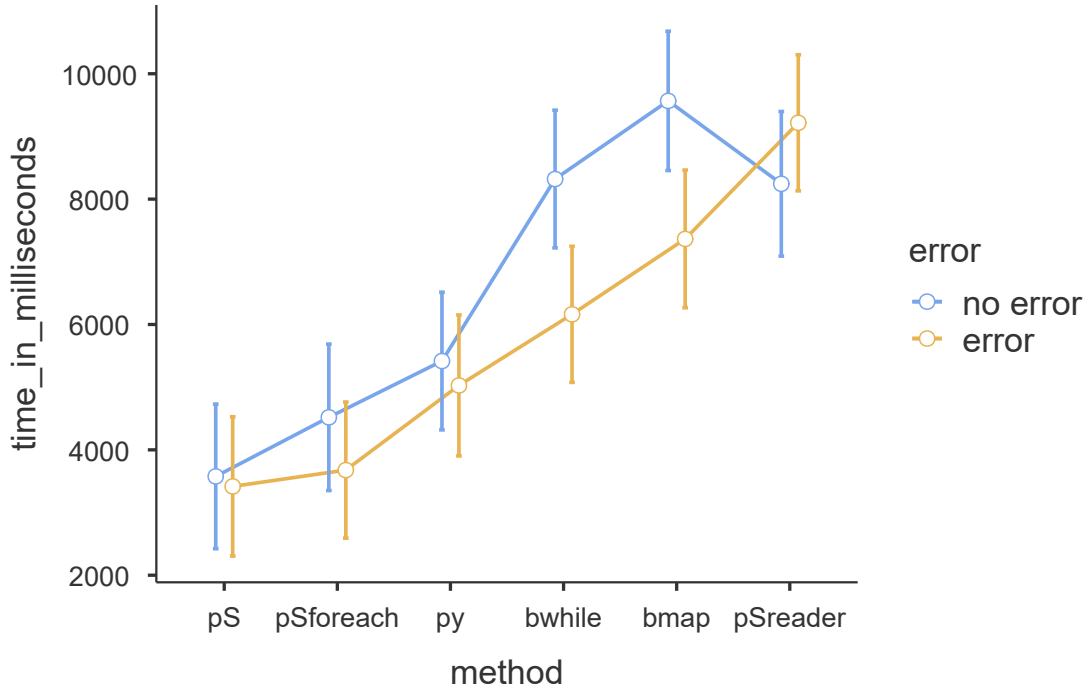


Figure 13: Interaction diagram method * error

1, PowerShell was the fastest. Participants 5 and 6 approach significance with $p = .059$ and $p = .054$, respectively.

Lastly, Table 15 shows the analysis of the pretask reading time data. Outliers outside the range $[Q1 - 1.5IQR, Q3 - 1.5IQR]$ were again excluded. The method had a large and significant effect on the pretask reading time ($p < .001$, $\eta_p^2 = .147$). The mean of the measured reading times was the highest for the `bwhile` method ($M = 3771$ ms), whereas the lowest was for the `py` method ($M = 1836$ ms). When the methods were grouped by language, another ANOVA revealed a significant but small effect of the programming language ($p < .001$, $\eta_p^2 = .055$; see Table 15). Bash pretasks had the highest mean reading time with 3351 milliseconds, whereas Python had the lowest with 1836 milliseconds.

Table 12: Results per participant without removing outliers

ANOVA on Reaction Time of method						
participant	p	η_p^2	N	TRT	$CI_{95\%}$	M
1	.006	.258	10	pS	2152; 5089	3621
			10	pSforeach	2969; 5777	4373
			10	py	3124; 9343	6234
			10	bwhile	4832; 15969	10400
			10	bmap	4725; 31573	18149
			10	pSreader	5495; 11412	8454
2	< .001	.428	10	pS	2799; 5582	4191
			10	pSforeach	3093; 7244	5169
			10	py	3481; 6930	5205
			10	bwhile	4562; 8629	6596
			10	bmap	5859; 11437	8648
			10	pSreader	8972; 13083	11027
3	< .001	.491	10	pS	3522; 7210	5366
			10	pSforeach	5352; 9191	7271
			10	py	4626; 8384	6505
			10	bwhile	9068; 13951	11509
			10	bmap	10946; 15222	13084
			10	pSreader	9085; 16338	12712
4	< .001	.345	10	pS	1739; 2632	2185
			10	pSforeach	1583; 2294	1939
			10	py	1182; 6532	3857
			10	bwhile	3151; 3895	3523
			10	bmap	3399; 5087	4243
			10	pSreader	4559; 6179	5369
5	< .001	.328	10	pS	1668; 2424	2046
			10	pSforeach	1915; 3419	2667
			10	py	2668; 5787	4228
			10	bwhile	3273; 5085	4179
			10	bmap	3038; 5325	4181
			10	pSreader	3610; 5899	4754
6	< .001	.603	10	pS	1861; 4051	2956
			10	pSforeach	1643; 3641	2642
			10	py	2838; 4393	3615
			10	bwhile	3774; 6132	4953
			10	bmap	5183; 7970	6577
			10	pSreader	6416; 9305	7861
7	.178	.129	10	pS	2348; 7993	5170
			10	pSforeach	2090; 9903	5997
			10	py	3108; 6087	4597
			10	bwhile	3900; 10393	7146
			10	bmap	3841; 9919	6880
			10	pSreader	3958; 17467	10712
8	< .001	.321	10	pS	2932; 4263	3598
			10	pSforeach	2381; 5895	4138
			10	py	4755; 8602	6678
			10	bwhile	3966; 7964	5965
			10	bmap	4248; 10698	7473
			10	pSreader	6583; 13900	10242
9	< .001	.471	10	pS	7474; 11720	9597
			10	pSforeach	7535; 13672	10603
			10	py	9492; 15301	12396
			10	bwhile	11871; 20183	16027
			10	bmap	16825; 25458	21142
			10	pSreader	15585; 22804	19194

Table 13: Results per participant removing outliers per participant

ANOVA on Reaction Time of method						
participant	p	η_p^2	TRT	$CI_{95\%}$	M	outliers
1	.005	.281	pS	2152; 5089	3621	0
			pSforeach	2969; 5777	4373	0
			py	3124; 9343	6234	0
			bwhile	4336; 12794	8565	1
			bmap	5504; 14180	9842	2
			pSreader	6245; 10067	8156	2
2	< .001	.602	pS	2799; 5582	4191	0
			pSforeach	2828; 6235	4532	1
			py	3535; 5571	4553	1
			bwhile	4332; 7574	5953	1
			bmap	6551; 10906	8728	2
			pSreader	8972; 13083	11027	0
3	< .001	.491	pS	3522; 7210	5366	0
			pSforeach	5352; 9191	7271	0
			py	4626; 8384	6505	0
			bwhile	9068; 13951	11509	0
			bmap	10946; 15222	13084	0
			pSreader	9085; 16338	12712	0
4	< .001	.780	pS	1671; 2442	2057	1
			pSforeach	1583; 2294	1939	0
			py	2152; 2822	2487	2
			bwhile	3151; 3895	3523	0
			bmap	3365; 4526	3946	1
			pSreader	4559; 6179	5369	0
5	< .001	.382	pS	1668; 2424	2046	0
			pSforeach	1915; 3419	2667	0
			py	2643; 4684	3663	1
			bwhile	3124; 4730	3927	1
			bmap	3038; 5325	4181	0
			pSreader	3610; 5899	4754	0
6	< .001	.747	pS	1766; 3420	2593	1
			pSforeach	1610; 2964	2287	1
			py	2838; 4393	3615	0
			bwhile	4403; 6258	5330	1
			bmap	5183; 7970	6577	0
			pSreader	7227; 9440	8334	1
7	.017	.235	pS	2655; 5435	4045	1
			pSforeach	3164; 4108	3636	2
			py	3108; 6087	4597	0
			bwhile	3900; 10393	7146	0
			bmap	3841; 9919	6880	0
			pSreader	5673; 10023	7848	1
8	< .001	.415	pS	2932; 4263	3598	0
			pSforeach	2381; 5895	4138	0
			py	4708; 7267	5988	1
			bwhile	3966; 7964	5965	0
			bmap	3914; 8957	6436	1
			pSreader	8171; 10482	9327	2
9	< .001	.471	pS	7474; 11720	9597	0
			pSforeach	7535; 13672	10603	0
			py	9492; 15301	12396	0
			bwhile	11871; 20183	16027	0
			bmap	16825; 25458	21142	0
			pSreader	15585; 22804	19194	0

Table 14: Results per participants on language

ANOVA on Reaction Time of language						
participant	p	η_p^2	N	TRT	$CI_{95\%}$	M
1	.004	.179	20	Bash	7471; 21078	14275
			30	PowerShell	4164; 6801	5483
			10	Python	3124; 9343	6234
2	.232	.050	20	Bash	5991; 9252	7622
			30	PowerShell	5312; 8279	6795
			10	Python	3481; 6930	5205
3	< .001	.235	20	Bash	10787; 13806	12297
			30	PowerShell	6686; 10213	8450
			10	Python	4626; 8384	6505
4	.404	.031	20	Bash	3433; 4333	3883
			30	PowerShell	2505; 3824	3164
			10	Python	1182; 6532	3857
5	.059	.094	20	Bash	3523; 4837	4180
			30	PowerShell	2552; 3759	3156
			10	Python	2668; 5787	4228
6	.054	.097	20	Bash	4855; 6674	5765
			30	PowerShell	3397; 5575	4486
			10	Python	2838; 4393	3615
7	.409	.031	20	Bash	5010; 9016	7013
			30	PowerShell	4708; 9878	7293
			10	Python	3108; 6087	4597
8	.796	.008	20	Bash	4972; 8465	6719
			30	PowerShell	4337; 7647	5992
			10	Python	4755; 8602	6678
9	.003	.181	20	Bash	15620; 21549	18584
			30	PowerShell	10906; 15357	13131
			10	Python	9492; 15301	12396

Table 15: Results of loops main experiment on Pretask Reading Time

ANOVA on Pretask Reading Time								
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M	
method	5	17.1	< .001	.147	pS	2101; 2942	2522	
					pSforeach	2276; 3187	2732	
					py	1543; 2129	1836	
					bwhile	3137; 4405	3771	
					bmap	2361; 3531	2946	
					pSreader	2030; 2826	2428	
ANOVA on Pretask Reading Time with language								
Variable	df	F	p	η_p^2	TRT	$CI_{95\%}$	M	$\Delta M\%$ (Python)
language	2	14.7	< .001	.055	Bash	2920; 3782	3351	+82.5%
					PowerShell	2320; 2806	2563	+39.6%
					Python	1543; 2129	1836	0%

9.6 Threats to Validity

This experiment faces threats to validity similar to those in the main experiment on conditionals. Threats related to sample size, outliers, and participants also apply to this experiment (see Section 8.6). Additionally, this experiment presents some unique threats.

Looping Methods Although the looping methods used for the experiment all attempt to accomplish the same task—iterating through an input file—they may not fairly represent looping in each programming language. For example, the `pSreader` method in PowerShell uses explicit stream handling, opening and closing a stream reader. While this is needed to iterate through a file, it deviates from just the looping logic and creates additional complexity. Additionally, given the many ways to loop through files in each language, drawing firm conclusions about the scripting language may be difficult just based on the chosen looping methods.

9.7 Discussion

The results in Table 9 indicate a significant effect of the looping method on reaction times, with a very large effect size ($p < .001$, $\eta_p^2 = .241$). The methods used can be seen in Figure 11. In the experiment, the `pS` method was the fastest, whereas `pSreader` was the slowest, with a mean reaction time 150.7% longer than `pS`. Consequently, PowerShell exhibits the greatest variability, encompassing both the fastest and slowest looping methods. Except for the `pSreader` method, both Bash methods—`bwhile` and `bmap`—were the slowest, with mean reaction times 106.9% and 141.9% longer than `pS`, respectively. Individual analyses for the participants generally confirm the overall results (see Tables 12 and 13). Across all participants, either `pSforeach` or `bmap` was the slowest looping method, regardless of whether outliers were included.

Grouping the methods by language (see Table 10) also revealed a significance of the language on the reaction time ($p < .001$, $\eta_p^2 = .078$). Python had the shortest mean reaction time, followed by PowerShell, which was 4.5% longer, and Bash, which was 49.9% longer than Python. Although language had a significant effect overall, individual analyses confirmed this significance for only three participants: participant 1, 3, and 9 (see Table 14). For these three participants, Bash was also the slowest language.

An analysis of the correctness of responses (see Table 9) showed that the looping method had a significant effect on the accuracy ($p = .002$). The `pS` and `pSforeach` methods from PowerShell produced the fewest incorrect answers, with 12 and 13 errors, respectively. In contrast, the `bmap` and `pSreader` methods produced the most incorrect answers, with 28 and 30 errors, respectively.

Notably, the ranking of methods by number of errors corresponds to their ranking by mean reaction time. This may indicate that some methods were more complex than others, leading participants to spend more time and make more errors when completing them. However, at the individual level, no participant showed a significant effect (see Table 11). We think that increasing the number of tasks may cause participants' results to align more closely with those observed across the full data set.

Lastly, we measured the time participants took to read the pretask for each method, i.e., to review the correct code snippet and possibly memorize key aspects (see Table 15). Here, the order of methods did not correspond to their ranking by reaction time. While a significant effect was observed ($p < .001$, $\eta_p^2 = .147$), `py` had the shortest mean pretask reading time among the methods. Grouping the methods by language revealed a significant effect of the language on the reading time ($p < .001$, $\eta_p^2 = .055$). Python required the least time to read, followed by PowerShell, which was 39.6% longer, and Bash, which took 82.5% longer than Python.

Across all measures, Bash consistently performed among the worst, with reaction times nearly 50% longer than those of Python and PowerShell, along with considerably longer pretask reading times. This further suggests the higher complexity of loops written in Bash.

10 Conclusion

This study used controlled experiments to investigate if the choice of scripting language—Bash, PowerShell, or Python—affects code readability and error detection in loops and conditional statements. Following three preliminary experiments, two main experiments were conducted, and data from nine participants were analyzed.

The results of the main experiment on conditional statements show a significant effect of the language on reaction times. Tasks in Python yielded the fastest responses, followed by PowerShell (29.7% slower) and Bash (55.3% slower). However, no significant effect of language was found on the number of errors made.

In the main experiment on loops, the looping method used significantly influenced reaction times. When grouping the methods by language, Python again led to the fastest responses, followed by PowerShell (4.5% slower) and Bash (49.9% slower). However, separate analyses for each participant only confirmed this for three of nine participants. We think that a larger sample size of tasks and participants may cause participants to produce results more consistent with the overall trend. We also measured the time participants spent reading the pretask, which consisted of an error-free code snippet for each method. The purpose of the pretask was for participants to review the code and potentially memorize key aspects before performing the main task. The results showed that Python required the least time to read, followed by PowerShell (39.6% longer), and Bash (82.5% longer).

Across both experiments and all measures, Bash consistently performed the worst, showing substantially longer reaction and pretask reading times. In our experiments, times may have been influenced by numerous uncontrolled factors, which complicates interpreting why Bash displayed lower readability. In Bash, incorrect spacing between tokens can cause errors or unexpected behavior, and quotations are required to prevent unintended word splitting. Additionally, Bash also uses both symbolic (e.g. `<`, `>`, and `==`) and word-like comparison operators (e.g. `-lt`, `-gt`, and `-eq`), adding further complexity. We argue that the variety of potential pitfalls in Bash contributes to its lower readability. Although, the results could reflect participants' lower experience with Bash, previous research provides supporting evidence for its inherent complexity. Dong et al. [12] found that among one million investigated GitHub repositories, only 20% of Bash scripts were free of code smells.

While these experiments provide valuable insights, they have potential limitations. For instance, the loops experiment employed specific looping methods to iterate through an input file, which may not represent all types fairly and could limit the generalizability of the findings. Nevertheless, the consistency

of results across languages and measures suggests that this work can serve as a foundation for future studies investigating readability and error detection in scripting languages.

References

- [1] 4. More Control Flow Tools. <https://docs.python.org/3/tutorial/controlflow.html>. (accessed 2025-08-15).
- [2] Bash Reference Manual. <https://www.gnu.org/software/bash/manual/bash.html>. (accessed 2025-08-15).
- [3] GitHub - splugrub/Experiment. <https://github.com/splugrub/Experiment>. (accessed 2025-08-24).
- [4] Jamovi - open statistical software for the desktop and cloud. <https://www.jamovi.org/>. (accessed 2025-08-14).
- [5] PYPL PopularitY of Programming Language index. <https://pypl.github.io/PYPL.html>. (accessed 2025-08-15).
- [6] Python (programming language) - CodeDocs. <https://codedocs.org/what-is/python-programming-language>. (accessed 2025-08-15).
- [7] Software Developers Statistics 2024 - State of Developer Ecosystem Report. <https://www.jetbrains.com/lp/devecosystem-2024>. (accessed 2025-08-15).
- [8] TIOBE Index - TIOBE. <https://www.tiobe.com/tiobe-index/>. (accessed 2025-08-23).
- [9] Charles Bellemare, Luc Bissonnette, and Sabine Kröger. Statistical Power of Within and Between-Subjects Designs in Economic Experiments, November 2014.
- [10] Angus M. Brown. A new software for carrying out one-way ANOVA post hoc tests. *Computer Methods and Programs in Biomedicine*, 79(1):89–95, July 2005.
- [11] Gary Charness, Uri Gneezy, and Michael A. Kuhn. Experimental methods: Between-subject and within-subject design. *Journal of Economic Behavior & Organization*, 81(1):1–8, January 2012.
- [12] Yiwen Dong, Zheyang Li, Yongqiang Tian, Chengnian Sun, Michael W. Godfrey, and Meiyappan Nagappan. Bash in the Wild: Language Usage, Code Smells, and Bugs. *ACM Trans. Softw. Eng. Methodol.*, 32(1):8:1–8:22, February 2023.
- [13] Michael Felderer and Guilherme Horta Travassos. The evolution of empirical methods in software engineering. In *Contemporary Empirical Methods in Software Engineering*. Springer, May 2020.

-
- [14] Stefan Hanenberg and Nils Mehlhorn. Two N-of-1 self-trials on readability differences between anonymous inner classes (AICs) and lambda expressions (LEs) on Java code snippets. *Empirical Software Engineering*, 27(2):33, December 2021.
- [15] Stefan Hanenberg, Johannes Morzeck, and Volker Gruhn. Indentation and reading time: A randomized control trial on the differences between generated indented and non-indented if-statements. *Empirical Software Engineering*, 29(5):134, August 2024.
- [16] John Johnson, Sergio Lubo, Nishitha Yedla, Jairo Aponte, and Bonita Sharif. An Empirical Study Assessing Source Code Readability in Comprehension. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 513–523, September 2019.
- [17] Natalia Juristo and Ana M Moreno. *Basics of Software Engineering Experimentation*. Springer Science & Business Media, 2013.
- [18] Alexander Kanavin. An overview of scripting languages. Master’s thesis, Lappeenranta University of Technology, Finland, 2002.
- [19] Khalid Khanfar, A Elzamly, W Al-Ahmad, E El-Qawasmeh, K Alsamara, and S Abuleil. Managing Software Project Risks with the Chi-Square (χ^2) Technique. *International Management Review*, 4:18–29, 2008.
- [20] Jessica Middlemis Maher, Jonathan C. Markey, and Diane Ebert-May. The Other Half of the Story: Effect Size Analysis in Quantitative Research. *CBE Life Sciences Education*, 12(3):345–351, 2013.
- [21] Microsoft. Everything you wanted to know about the if statement - PowerShell. <https://learn.microsoft.com/en-us/powershell/scripting/learn/deep-dives/everything-about-if?view=powershell-7.5>. (accessed 2025-08-15).
- [22] Microsoft. Flow control - PowerShell. <https://learn.microsoft.com/en-us/powershell/scripting/learn/ps101/06-flow-control?view=powershell-7.5>. (accessed 2025-08-15).
- [23] Microsoft. What is PowerShell? - PowerShell. <https://learn.microsoft.com/en-us/powershell/scripting/overview?view=powershell-7.5>. (accessed 2025-08-14).
- [24] Cameron Newham. *Learning the Bash Shell: Unix Shell Programming*. O’Reilly Media, Inc., March 2005.
- [25] J.K. Ousterhout. Scripting: Higher level programming for the 21st Century. *Computer*, 31(3):23–30, March 1998.

-
- [26] Nancy Pennington. Stimulus structures and mental representations in expert comprehension of computer programs. *Cognitive Psychology*, 19(3):295–341, July 1987.
- [27] Chet Ramey et al. Bash reference manual. *Network Theory Limited*, 15, 1998.
- [28] Baishakhi Ray, Daryl Posnett, Vladimir Filkov, and Premkumar Devanbu. A large scale study of programming languages and code quality in github. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2014, pages 155–165, New York, NY, USA, November 2014. Association for Computing Machinery.
- [29] John T. E. Richardson. Eta squared and partial eta squared as measures of effect size in educational research. *Educational Research Review*, 6(2):135–147, January 2011.
- [30] Michael Shepard. *Getting Started with Powershell*. Packt Publishing Ltd, 2015.
- [31] Forrest Shull, Janice Singer, and Dag IK Sjøberg. *Guide to Advanced Empirical Software Engineering*. Springer, 2008.
- [32] Andreas Stefik, Stefan Hanenberg, Mark McKenney, Anneliese Andrews, Srinivas Kalyan Yellanki, and Susanna Siebert. What is the foundation of evidence of human factors decisions in language design? an empirical study on programming language workshops. In *Proceedings of the 22nd International Conference on Program Comprehension*, ICPC 2014, pages 223–231, New York, NY, USA, June 2014. Association for Computing Machinery.
- [33] Harald O. Stolberg, Geoffrey Norman, and Isabelle Trop. Randomized Controlled Trials. *American Journal of Roentgenology*, 183(6):1539–1544, December 2004.
- [34] H. P. Vinutha, B. Poornima, and B. M. Sagar. Detection of Outliers Using Interquartile Range Technique from Intrusion Dataset. In Suresh Chandra Satapathy, Joao Manuel R.S. Tavares, Vikrant Bhateja, and J. R. Mohanty, editors, *Information and Decision Sciences*, pages 511–518, Singapore, 2018. Springer.
- [35] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, et al. *Experimentation in Software Engineering*, volume 236. Springer, 2012.

A Contents of the USB flash drive

The USB flash drive included in every physical copy contains the following:

- A digital copy of this thesis as a PDF document
- The collected data for each participant
- The .omv files for the analysis in jamovi
- The source code containing all experiments

Eidesstattliche Versicherung

Hiermit versichere ich an Eides Statt, dass ich diese Arbeit bzw. im Fall einer Gruppenarbeit den von mir entsprechend gekennzeichneten Anteil an der Arbeit selbständig verfasst habe. Ich habe keine unzulässige Hilfe Dritter in Anspruch genommen. Zudem habe ich keine anderen als die angegebenen Quellen und Hilfsmittel benutzt und alle Ausführungen (insbesondere Zitate), die anderen Quellen wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht.

Ich versichere, dass die von mir in elektronischer Form eingereichte Version dieser Arbeit mit den eingereichten gedruckten Exemplaren übereinstimmt.

Mir ist bekannt, dass im Falle eines Täuschungsversuches die betreffende Leistung als mit "nicht ausreichend" (5,0) bewertet gilt. Zudem kann ein Täuschungsversuch als Ordnungswidrigkeit mit einer Geldbuße von bis zu 50.000 Euro geahndet werden. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuchs kann ich zudem exmatrikuliert werden.

Mir ist bekannt, dass sich die Prüferin oder der Prüfer bzw. der Prüfungsausschuss zur Feststellung der Täuschung des Einsatzes einer entsprechenden Software oder sonstiger elektronischer Hilfsmittel bedienen kann.

Ich versichere an Eides Statt, dass ich die vorbenannten Angaben nach bestem Wissen und Gewissen gemacht habe und dass die Angaben der Wahrheit entsprechen und ich nichts verschwiegen habe.

Die Strafbarkeit einer falschen eidesstattlichen Versicherung ist mir bekannt, insbesondere die Strafandrohung gemäß §§ 156, 161 StGB, auf welche ich konkret hingewiesen wurde.

§ 156 Falsche Versicherung an Eides Statt

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei Jahren oder mit Geldstrafe bestraft.

§ 161 Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Straflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtigt. Die Vorschriften des § 158 Abs. 2 und 3 gelten entsprechend.

Ort, Datum

Unterschrift